分类号 _____ 密 级 _____

U D C _____ 编 号 10486

武漢大學

硕 士 专 业 学 位 论 文

基于层内分组和层间选择的 Transformer 优化技术研究

研 究 生 姓 名 ： 武 涵

学 号 ： 2021282110122

指导教师姓名、职称 ： 彭 敏 教授

专业类别（领域） ：人工智能（自然语言处理）

二〇二三年五月

Research on Transformer Optimization Technology Based on Intra-Layer Clustering and Inter-Layer Selection

By

Wu Han

Supervised By

Prof. Peng Min

Wuhan University

May, 2023

论文原创性声明

本人郑重声明：所呈交的学位论文，是本人在导师指导下，独立进行研究工作所取得的研究成果。除文中已经标明引用的内容外，本论文不包含任何其他个人或集体已发表或撰写的研究成果。对本文的研究做出贡献的个人和集体，均已在文中以明确方式标明。本声明的法律结果由本人承担。

学位论文作者（签名）：_____

日期：　　年　　月　　日

摘要

近年来，深度学习技术飞速发展，已经在诸多领域取得了革命性的突破。在深度学习领域中，基于注意力机制的 Transformer 是一个非常有效的建模框架，在此基础上衍生发展出了许多具有代表性的模型，ChatGPT 更是有引发新一轮 AI 变革的势头。Transformer 模型的有效性很大程度上来源于注意力机制对于全局信息依赖的捕获，然而注意力计算也带来了平方级$O(L^2d)$的复杂度。神经网络规模和参数量的增大对计算和内存资源提出了需求，这也限制了模型在长序列数据场景和计算资源受限场景的深度应用，如何对 Transformer 模型进行优化加速是一个亟待解决的关键问题。现有的一些研究工作基于稀疏的思想，使用了稀疏注意力的方法来提升模型效率。然而基于位置的稀疏注意力未能考虑样本的差异化信息，部分基于内容的稀疏注意力存在分组误差和信息交互损失问题，影响模型效果。此外，这些方法仅从单一横向层内视角进行优化设计，缺少对整个模型结构其他方面的优化思考。因此，在模型效果和模型效率上都还存在一定的优化空间。

本文以 Transformer 模型结构优化为目标，提出了基于层内分组和层间选择的综合优化加速方案 CSformer。主要包括：1）在 Transformer 模型多层堆叠编码器或解码器的单独一层内部对序列进行聚类分组，采用基于内容的稀疏注意力代替完整注意力计算。在分组的不同阶段提出了聚类中心向量生成、聚类中心信息增强和分组损失计算等方法，来提高分组的准确性。2）提出了分类信息层次融合的方法，以优化分组注意力模型在分类任务上的表现。3）在模型编码器或解码器层之间以及不同的分组之间，首先为了衡量 Token 的贡献度，提出了三个基于注意力分数的计算策略。基于这些策略，在组与组之间提出了全局信息引入方法，用于弥补分组后的信息交互损失；在层与层之间提出了序列长度缩减方法，用于对 Token 进行选择与丢弃，舍弃不重要的计算以获得更高的效率。

本文在文本分类和词分类两个任务的多个数据集上开展了实验。实验结果表明，本文提出的 CSformer 模型方案在训练的收敛速度上比基线模型更快，表现出的模型效果和模型效率也普遍优于基线模型，在内存消耗减少 22.66%、时间消耗减少 37.98%的同时，模型性能提升了 9.58%。此外，消融实验和可视化实验的结果也验证了本文所提出相关策略的有效性和高效性。通过调整序列长度缩减比例、全局信息数量等相关参数，还可根据实际场景的需求在模型效率和效果之间进行侧重选择，有助于更好地落地于具体应用。

关键词：优化加速；分类；层内分组；层间选择

Abstract

In recent years, there has been a rapid development of deep learning techniques, resulting in revolutionary breakthroughs in various fields. Within the field of deep learning, the Transformer model based on attention mechanisms has proven to be a highly effective modeling framework. This has led to the development of many representative models, with ChatGPT in particular sparking a new wave of AI revolution. The effectiveness of the Transformer model is largely due to the attention mechanism's ability to capture global information dependencies. However, attention calculations also bring a complexity of $O(L^2d)$. As network size and parameter count increase, there are growing demands on computational and memory resources, which limits the model's depth of application in long sequence data and resource-limited scenarios. Therefore, optimizing and accelerating the Transformer model has become an urgent problem that needs to be addressed. Existing research has explored the use of sparse attention to improve model efficiency. However, position-based sparse attention fails to consider differentiated information among samples, while content-based sparse attention may suffer from clustering errors and loss of information interaction, which can affect model performance. Additionally, these methods only optimize and design based on a single lateral layer dimension and lack optimization considerations for other dimensions of the overall model structure. As a result, there is still room for improvement in both model effectiveness and efficiency.

This thesis aims to optimize the Transformer model structure and proposes a comprehensive optimization and acceleration scheme, CSformer, based on intra-layer clustering and inter-layer selection. The scheme includes the following main components: 1) intra-layer clustering and clustering of sequences within a single layer of the stacked encoder or decoder of the Transformer model, which replaces complete attention calculation with content-based sparse attention. To improve the accuracy of clustering, several methods are proposed for cluster center vector generation, cluster center information enhancement, and cluster loss calculation at different stages of clustering. 2) A hierarchical fusion method for classification information is proposed to optimize the performance of the clustering attention model in classification tasks. 3) For measuring the contribution of tokens, three token contribution calculation strategies based on attention scores are proposed between the model encoder or decoder layers and between different clusters. Based on these strategies, a global

information introduction method is proposed between clusters to compensate for the loss of information interaction caused by clustering, and a sequence length reduction method is proposed between layers to select and discard tokens according to their contribution and to achieve higher efficiency by discarding unimportant computations.

This thesis conducts experiments on multiple datasets for both text and word classification tasks. The experimental results indicate that the proposed CSformer model scheme shows faster convergence speed than the baseline model during training, and generally performs better in terms of model effectiveness and efficiency. The model performance improved by 9.58%, while memory consumption decreased by 22.66% and time consumption decreased by 37.98%. Additionally, the results of ablation experiments and visualization experiments confirm the effectiveness and efficiency of the proposed strategies. By adjusting relevant parameters such as sequence length reduction ratio and global information quantity, the model can be customized to better fit the requirements of specific applications in terms of efficiency and effectiveness.

Keywords: Optimization and Acceleration, Classification, Intra-Layer Clustering, Inter-Layer Selection

目 录

摘要 I
Abstract II
第 1 章 绪论 1
1.1 研究背景与意义 1
1.2 国内外研究现状 2
1.2.1 模型计算效率优化 2
1.2.2 模型工程化加速 4
1.3 主要研究内容 5
1.4 论文的组织结构 6
第 2 章 相关理论和技术 9
2.1 Transformer 模型架构 9
2.1.1 主要模块及应用方式 9
2.1.2 模型复杂度和参数量级 12
2.2 模型通用优化技术 12
2.2.1 模型训练部署加速 13
2.2.2 框架层优化 14
2.3 Transformer 模型优化加速技术 14
2.3.1 针对注意力模块的稀疏优化 14
2.3.2 针对 Token 的选择优化 16
2.3.3 针对模型参数的剪枝优化 17
2.4 本章小结 18
第 3 章 基于层内分组的稀疏注意力策略 19
3.1 研究动机与方法概述 19
3.2 Token 分组注意力流程 20
3.2.1 分组视野范围 20
3.2.2 聚类中心向量的生成 20
3.2.3 基于注意力的分组计算 23
3.2.4 组内注意力计算 25
3.2.5 注意力分组的修正 25
3.3 基于分类信息层次融合的注意力结构扩充 27
3.4 复杂度分析 28
3.5 本章小结 29

第 4 章 基于贡献度的 Token 层间选择策略 30
4.1 研究动机与方法概述 30
4.2 Token 贡献度评估策略 31
4.2.1 Token 的组内贡献度 31
4.2.2 Token 对中心向量的贡献度 32
4.2.3 Token 对分类向量的贡献度 32
4.3 局部和全局注意力的信息融合 33
4.4 层间的序列长度缩减 35
4.5 复杂度分析 36
4.6 本章小结 37
第 5 章 实验设计与分析 38
5.1 实验设置 38
5.1.1 基线模型 38
5.1.2 实验任务和数据集 39
5.1.3 参数设置 40
5.1.4 评估指标 41
5.1.5 实验环境 41
5.2 语言分类任务中的性能与分析 42
5.2.1 文本分类任务 42
5.2.2 词级别分类任务 45
5.3 消融实验 46
5.3.1 聚类中心生成算法 47
5.3.2 全局信息引入策略 47
5.3.3 分组误差弥补的策略组合 48
5.4 参数实验 48
5.4.1 视野范围大小 48
5.4.2 序列长度缩减比例 49
5.4.3 输入序列的长度 50
5.5 可视化实验 51
5.6 本章小结 53
第 6 章 总结与展望 54
6.1 本文总结 54
6.2 未来展望 54
参考文献 56

致谢..... 61
附录 A：在学期间参与的科研项目 62

第 1 章 绪论

1.1 研究背景与意义

近年来，人工智能和深度学习等技术飞速发展。在深度学习领域，基于注意力机制的 Transformer[1]是一个非常有效的建模框架。借助互联网高速发展所产生的海量数据，基于该框架已经发展出了许多具有代表性的大模型，如自然语言处理领域的 BERT[2]、GPT-3[3]、T5[4]，面向计算机视觉的 ViT[5]、YOLO[6]、Swin[7]，面向音频处理的 Wave2Vec[8]、Conformer[9]等。这些模型能够从数据中学习到更多的信息，在诸多领域取得了革命性的突破，现如今已被广泛应用于各种实际场景，比如机器翻译、对话问答、文本分类、目标检测和视频修复等。基于 Transformer 架构的 ChatGPT[1]更是有引发新一轮 AI 变革的势头。

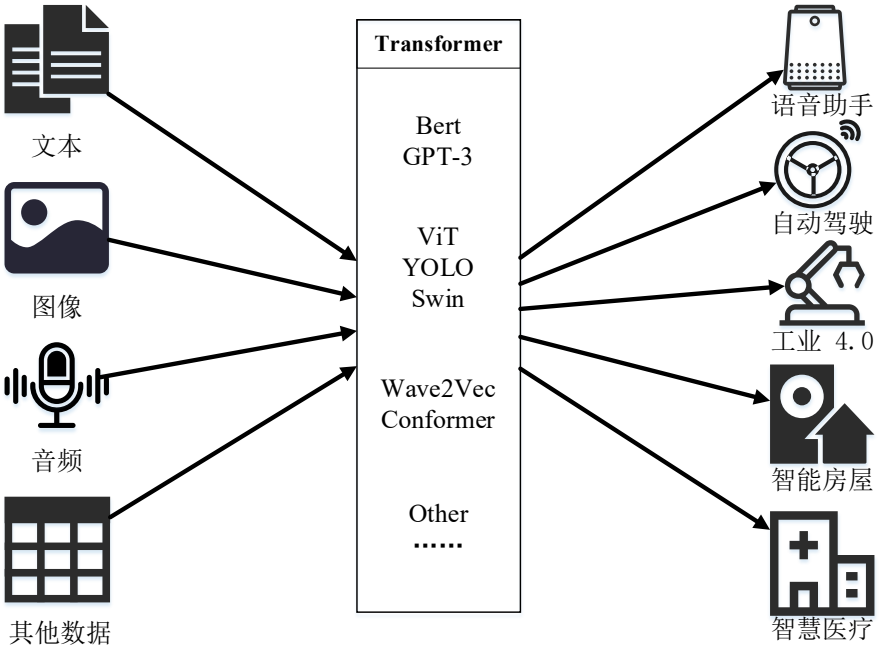


图 1.1 Transformer 架构模型的应用场景

随着深度神经网络在各种应用上取得越来越理想的效果，网络的规模也愈发庞大、层数越来越深，参数量也越来越多。Transformer 模型架构本身被设计为多层堆叠结构，模型参数量相较于其他模型自然要更大。基于 Transformer 的语言模型的参数量越来越大，从最初 BERT 的亿级参数增长到 GPT-3 的千亿级，Switch Transformer[10]甚至达到了万亿级参数。随着新模型的不断提出，这个数据量级还在继续增长。巨大的参数量和深层的结构使得神经网络能够从海量的数据

[1] https://openai.com/blog/chatgpt

中学习到更丰富的特征信息。同时这也对硬件设备当前的能力和未来的发展提出了更高的要求，进而促进了对深度神经网络进行压缩和效率优化的研究进展。

Transformer 模型架构的有效性很大程度上来源于注意力机制对于全局信息依赖的捕获。然而，需要注意的一个问题是，注意力机制的计算复杂度为平方级别$O(L^2d)$，其中L是序列的长度，d是隐藏层特征向量的维度。该核心模块带来了计算资源消耗和内存占用的问题，限制了模型在长序列数据场景和计算资源受限设备（如移动设备、IoT）上的深度应用。随着 ChatGPT 引领的大模型发展浪潮，Transformer 的效率优化问题就变得更为重要。除此之外，模型训练推理过程的计算导致的过度能源消耗和环境影响问题也引起了广泛的关注。

综上所述，深度学习模型的高速发展与硬件的迭代更新之间存在着不匹配的问题。因此，从模型层面入手对模型结构进行优化，是近些年学术界的一个研究热点，也是工业界一个亟待解决的关键问题。语言理解的常识认知中包含许多有价值的现象：一句话内的词语间相关性更高，不同句子中的词语之间相关性要弱很多，只有少数存在相关性；一句话中并非每个单词都对整句话的信息偏向有关键影响。在计算机视觉等领域也存在类似的先验现象。合理可行的优化策略需要参考场景本身存在的信息特征，设计特定的处理方法。在尽可能保证模型效果的前提下，减少模型内存占用并提升模型训练和推理过程的速度，助力基于 Transformer 架构的模型更好地落地于各种应用场景。

1.2 国内外研究现状

基于 Transformer 架构的模型在多个领域的许多任务上都取得了令人满意的效果，是近些年深度学习领域一个主流的研究方向。随着国内外学者对模型复杂度增长和硬件发展不匹配问题的重视，许多工作也开始聚焦于提升模型效率。针对该问题，一些工作[11][12][13]从模型可解释性角度出发探究模型的工作机制。根据从模型参数中观察到的现象和结论，提出了针对性的优化策略。在优化策略上又可以分为模型结构特定的效率优化以及模型结构无关的工程化加速技术。

1.2.1 模型计算效率优化

针对 Transformer 模型对计算资源和内存要求过高的问题，许多研究工作尝试改造原始的 Transformer 结构，提出了各种模型优化变种以期望提高模型效率。模型的效率有很多参考指标，可以是模型的内存占用，也可以是计算成本，或者是其他的评估指标，本文主要从计算消耗和内存占用两方面展开分析。目前主流的模型效率优化方案大概可以分为如下几种分支。

基于稀疏模式的注意力近似计算。近年来的一些研究工作发现注意力矩阵呈

现稀疏分布的特征，因此提出了多种稀疏模式应用于注意力计算过程，由此限制 Query 所能关注到的 Key-Value 对的范围和数量，以达到减少计算的目的。LoBART[14]、Longformer[15]等提出了局部模式稀疏化策略，使得注意力机制只作用在固定模式下的 Token 之间。然而这种固定的模式未能考虑到不同样本的差异化信息，同时还形成了不同位置的信息交互壁垒，进而导致在模型效果上有所下降。Big Bird[16]等工作以组合的思想为核心，使用多种访问模式分别从全局和局部来捕获不同层面的信息，扩大了注意力机制的视野覆盖域。然而其稀疏模式仍然是固定的，未能考虑不同 Token 之间的相关性。在计算层面，基于固定稀疏模式的注意力机制往往需要目标硬件设备的支持才能够进行高效计算，而目前的硬件一般都没有对稀疏注意力计算进行原生适配，这限制了部分稀疏模式的通用性。

近年来还有一些可学习的稀疏模式被提出，这些模式旨在以数据驱动的方式进行学习，可以根据 Token 之间的相关性对输入序列进行重排列。Reformer[17]引入了一种基于局部哈希敏感（Locality Sensitive Hashing）的相似性度量方法，以有效地将 Token 聚类成块，每个聚类内部独立进行注意力计算。与此类似的是，Routing Transformer[18]使用 K-Means 聚类算法将序列中的词分到不同集群中，然后将注意力计算的范围限制在独立的集群内。在这种方法下，模型的核心计算复杂度被降低到了$O(N\sqrt{N}d)$。然而这种分组方式的分组计算过程只起到划分的作用，它与整个神经网络的训练是相互独立的。其次，上述两种策略的分组计算过程分别使用随机投影和负欧式距离的方法作为输入向量相似度的指标，这与标准注意力机制的内积计算是不同的。综合来看，这些工作的合理性和准确性还有待商榷，其在信息交互过程产生的损失也是客观存在的问题。ClusterFormer[19]提出了神经聚类的方法，设定聚类中心由输入序列所有 Token 的隐藏状态进行加权更新，同时将聚类中心和序列 Token 的隐藏状态内积作为相似度指标来划分不同簇群。该方法确实在分组的处理上提出了一个较为合理的方案，然而在初始中心选取、分组信息来源和组外信息补充方面都还存在诸多问题，这种分组方案带来的误差也是个不容忽视的问题。

基于计算模拟的注意力近似。Linformer[20]巧妙地通过数学的方法重写了注意力机制，以避免显式地计算注意力矩阵。FNet[21]使用傅里叶计算的方法代替了注意力机制，在计算效率上也进行了提高。这些方法虽然计算上更为高效，但是对注意力机制的核心计算逻辑进行了修改，背离了后者设计的初衷。

针对参数的修剪算法。HAO Y 等人[13]提出了一种计算策略，用来衡量注意力机制中每对 Token 连接的贡献度，基于对实验结果的观察结论提出了新的注意力头空间修剪算法。此外对信息交互树进行了可视化呈现，以帮助更好地了解注意力机制的核心工作原理。Zhang 等人[11]对模型中权重参数的重要性和不确定性

进行了评估计算，根据重要性分数对模型进行了修剪。特别的，对于重要性分数低但不确定性高的权重进行了保留以保证训练的稳定性，提高模型泛化能力。然而，深度学习模型通常有大量参数，在训练起始阶段难以确定修剪的参数。并且修剪参数后模型结构发生变化，需要重新训练才能得到新的参数。因此，该策略只对模型推理过程有提升作用，在训练过程的效率提升不明显。

1.2.2 模型工程化加速

除了针对模型结构进行优化设计，还有一些不特定于模型结构的工程化加速技术，本文也选取了几种代表性策略进行了简要介绍。

模型剪枝。深度神经网络的模型参数存在冗余，因此一些研究工作[22][23][24]通过预定义剪枝和随机剪枝等方法对稠密模型的权重参数和通道进行了修剪，以此减少模型的参数数量和计算量。预定义剪枝可以进一步分为权重级剪枝、单元级剪枝和结构化剪枝等，不同策略中目标对象的粒度不同，但核心思想都是移除网络中非关键的冗余部分。随机剪枝不需要预定义标准，而是直接随机移除一定比例的权重或通道。国内外关于模型剪枝技术的研究已经非常成熟，不过该技术需要在模型训练好的基础上进行应用。

混合精度技术。该技术也是现如今比较成熟的方案，Micikevicius 等人[25]较早使用 IEEE 半精度格式存储神经网络中的权重、激活值和梯度，同时使用额外的技术来处理低精度的信息损失问题，将模型的内存消耗减少近 2 倍。Hoffer 等人[26]提出有效的归一化方法，显著改善了低精度模型训练中的稳定性和精度。

数值量化。一种将浮点计算转成低比特定点计算的技术，使用更少的比特位表示模型参数可以减少计算消耗和内存占用。黄[27]提出了一种新的 8 位浮点数格式 QFP8，以更精确地表示服从高斯分布的数值，在训练后量化、量化感知训练任务上均可以达到更高的推理精度。Zhang[28]等人提出了学习式的量化方法，联合训练了一个量化的、与比特操作兼容的量化器，适用于具有任意比特精度的网络权重和激活。结合硬件层面的协同实现，上述量化方法可以获得计算性能的提升，但在模型效果上往往会带来不可避免的下降。

知识蒸馏。知识蒸馏先训练一个大模型（称为教师网络），再以大模型的预测输出作为指导训练一个小模型（称为学生网络），以此将教师网络中学习到的知识迁移到学生网络中。Sanh 等人[29]在句子级别进行软标签和注意力矩阵两种方式的蒸馏，实现了在保留模型 97%语言理解能力的同时，参数量减少 40%、速度快 60%。该论文是知识蒸馏应用于自然语言处理领域的代表性工作，然而该策略重点在于解决模型应用部署上的问题，在训练阶段仍需要使用大量数据、投入许多计算资源来对教师网络进行训练。

1.3 主要研究内容

本文聚焦于“Transformer 模型的效率优化”这一问题，以模型参数可视化现象和部分可解释理论为指导，提出一系列模型结构层面的优化加速策略。旨在保证模型效果的同时，尽可能地减少计算和内存资源的消耗、提高模型效率。本文的研究内容分为如下几点：

（1）Transformer 模型横向层内的结构优化

在 Transformer 模型每一层编码器或解码器的内部（后续简称为横向层内），对序列进行聚类分组处理，以此实现稀疏注意力计算。基于分组的核心思想，在分组过程的不同时期设计多种综合策略，以提高分组的合理性与准确性。主要包括：1）在分组的前期阶段，通过预训练词向量和 Token 之间的距离等信息来计算序列中 Token 的相似度，以更好地利用词法和语义等多层面的信息对输入序列更准确地进行分组。同时，对于聚类中心向量的初始化过程，提出基于距离的平均生成策略和基于 K-Means++[30]的采样生成策略，以此将输入序列的信息融入到初始聚类中心中，进一步提高注意力分组的准确性。2）在分组的后期阶段，提出聚类中心信息增强策略和注意力分组损失训练方法，以此弥补分组过程的误差。通过上述不同阶段策略组合的分组处理，注意力信息交互的范围被限制在强相关的一组 Token 内部，这不仅可以减少模型整体的计算量，同时各组的组内注意力计算也可以并行执行，提高注意力机制的效率。

（2）面向分类任务的信息层次融合

在面向分类任务时，先前基于分组注意力的模型缺乏一个全局的分类标识和序列所有位置进行注意力交互以获取整体信息。针对该问题，提出一个分类信息层次融合的方法，通过“组内 Token 到聚类中心”和“聚类中心到全局分类向量”两个阶段的信息融合过程，使得整个输入序列的抽象信息最终汇聚到全局分类向量中，并通过该向量进行分类预测任务。

（3）Transformer 模型纵向层间的结构优化

在 Transformer 模型相邻的编码器或解码器之间以及不同分组之间（后续简称为纵向层间），首先设计多种任务特定的贡献度计算方法，以此对 Token 重要程度进行量化。在贡献度计算方法的基础上，提出两种应用策略：1）在组与组之间，提出全局信息引入策略。以 Token 和组内其他 Token 的注意力分数之和作为贡献度指标，在每个分组内部选取组内贡献度最大的 Token，将其作为全局信息的代表参与到各组的组内注意力计算过程。使得 Token 可以同时获取组内局部信息和组外全局信息，局部和全局综合对信息交互的损失进行弥补，提高模型效果。2）在层与层之间，提出序列长度缩减策略。以 Token 对全局分类向量的贡

献度作为指标进行排序，选择重要的 Token 进行保留，不重要的信息进行丢弃，以此来逐步缩减序列长度，进而节省计算资源和内存占用，提高模型效率。

综合层内分组和层间选择两个视角的优化策略，在模型效率和模型效果之间进行平衡，可根据应用场景的需求侧重达到理想的结果。综上所述，本文的创新点总结如下：

（1）在模型的横向层内，使用了基于分组思想的稀疏注意力策略。在分组前期，使用距离信息、预训练词向量信息等作为计算依据。同时提出了基于距离的平均生成算法和基于 K-Means++的采样生成算法对聚类中心的初始化过程进行优化。在分组后期，提出了聚类中心信息增强策略和分组损失计算方法，以此对分组误差进行弥补。多种策略综合以提高分组阶段的准确性，在提升模型效率的同时尽可能保证模型效果。

（2）提出了分类信息层次融合的方法，更有效、更合理地将输入序列中的分类信息融合进全局分类向量中，优化基于分组的稀疏注意力模型在面向分类任务时的应用过程。

（3）在模型的纵向层间，针对不同场景设计了特定的贡献度评估方法。基于贡献度评估方法，提出了全局信息引入策略和序列长度缩减策略。前者选择每组重要的 Token 作为全局信息引入到局部注意力计算中，弥补分组注意力带来的信息交互损失。后者依据 Token 对全局分类向量的贡献度进行选择与丢弃，缩减序列长度，提高模型的整体效率。

（4）实验结果表明，本文的模型方案在模型效果和模型效率上相比于基线模型普遍有一定提升，并且在模型收敛速度上也要优于基线模型。本文的研究工作为基于 Transformer 架构的模型提供了模型效率更高，并且保证模型效果的优化策略。通过调整相关参数，还可根据实际场景的需求在模型效率和效果之间进行侧重选择，有助于模型更好地进行落地应用。

1.4 论文的组织结构

本文共分为六章，整体组织结构如图 1.2 所示。

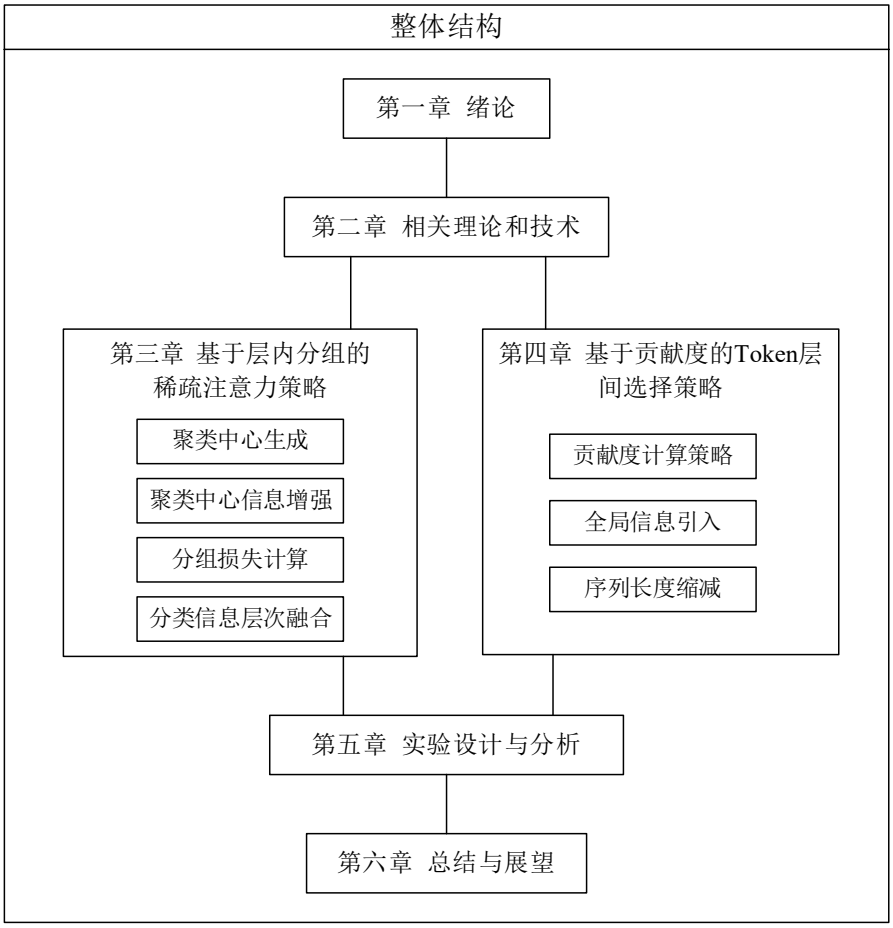


图 1.2 本文整体组织结构

本文每个章节的内容组织安排如下：

第一章为绪论。首先介绍本研究课题的背景和意义，基于 Transformer 架构的模型发展潜力巨大，但受限于平方级别的复杂度，在部分硬件设备和特殊场景下难以广泛开展应用；然后针对模型的优化加速问题梳理国内外相关研究分支和研究现状，总结现有研究工作存在的一些问题；最后介绍本文的研究内容和行文组织结构。

第二章为相关理论和技术，主要介绍与本文研究内容相关的一些理论背景和相关知识。具体包括 Transformer 模型架构、模型通用优化技术、Transformer 模型优化加速技术等。这些内容是第三、四章的理论基础，能够帮助读者更好地理解本文的研究内容和相关方法。

第三章介绍本文在横向视角提出的基于层内分组的稀疏注意力策略。具体内容以整体的处理流程为主线，同时包括初始聚类中心生成、分组信息补充、聚类中心信息增强、分组损失计算和分类信息层次融合等方面的相关方法。

第四章介绍本文在纵向视角提出的基于贡献度的 Token 层间选择策略。首先介绍本文设计的多种贡献度评估方法，为后续的选择处理过程提供了参考依据。

随后介绍基于贡献度评估方法的两种应用策略：全局信息补充和序列长度缩减，两者分别用于提高模型效果和模型效率。

第五章为实验设计与分析。首先介绍本文的实验基准、数据集、参数设置和实验环境等；然后针对各个数据集上的实验结果对不同模型的效果和效率进行对比，论证本文相关方法的有效性；其次，对相关模块进行消融实验，通过控制实验设置证实相关模块的有效性；接着，针对部分实验参数设置开展参数实验，通过实验结果分析不同参数对模型效率和模型效果的影响，同时得出较为推荐的实验参数设置。最后，通过可视化实验验证相关假设的合理性和方案的有效性。

最后一章为总结与展望。总结部分对本文的研究贡献和创新点进行总结回顾；展望部分对本文工作存在的不足进行反思，同时对后续可能的改进方向进行展望。

第 2 章 相关理论和技术

Transformer 模型架构是自然语言处理和计算机视觉等领域中许多模型的基础，也是本文的主要研究对象。本章首先详细介绍 Transformer 模型架构的关键模块和主要工作流程，基于此对其复杂度和参数量进行理论分析。深度学习模型的生命周期涉及到模型层设计、框架层实现和硬件层执行等多方面流程，本章随后按照视角从大到小，分别梳理模型通用优化策略和 Transformer 结构优化两个主要方向的相关理论和技术。这些内容是后续工作的基础，可以帮助更好地理解本文第三章和第四章的工作。

2.1 Transformer 模型架构

2.1.1 主要模块及应用方式

Transformer 是一个序列到序列的模型，由编码器和解码器两个主要模块组成，这两个模块都是由多个相同的子模块堆叠而成的多层结构。两个模块通常有如下三种组合使用方式：

（1）编码器和解码器两者同时使用时，是一个完整的模型结构。这种结构通常用于序列到序列的任务，比如神经机器翻译等。

（2）当只使用编码器时，输入序列经过多层模块的注意力交互，在最后一层得到包含上下文信息的输出，通常用于分类或序列标注任务。

（3）当只使用解码器时，从编码器获取信息的交叉注意力模块也被移除，模型的输入是来自目标序列的标记序列，通常用于序列生成语言建模等。

每个编码器子模块主要由多头自注意力模块和前馈神经网络两部分组成。为了构建层数更多的深层次模型，每个模块的输入和输出会通过残差的形式进行连接，然后再进行层归一化处理得到子模块的输出。与编码器相比，解码器在多头自注意力模块和前馈神经网络之间还额外加入了交叉注意力模块。除此之外，解码器还引入了特别的掩蔽机制，用以防止每个位置的 Token 和该位置后面的 Token 产生注意力信息交互。

标准 Transformer 的整体结构如图 2.1 所示。

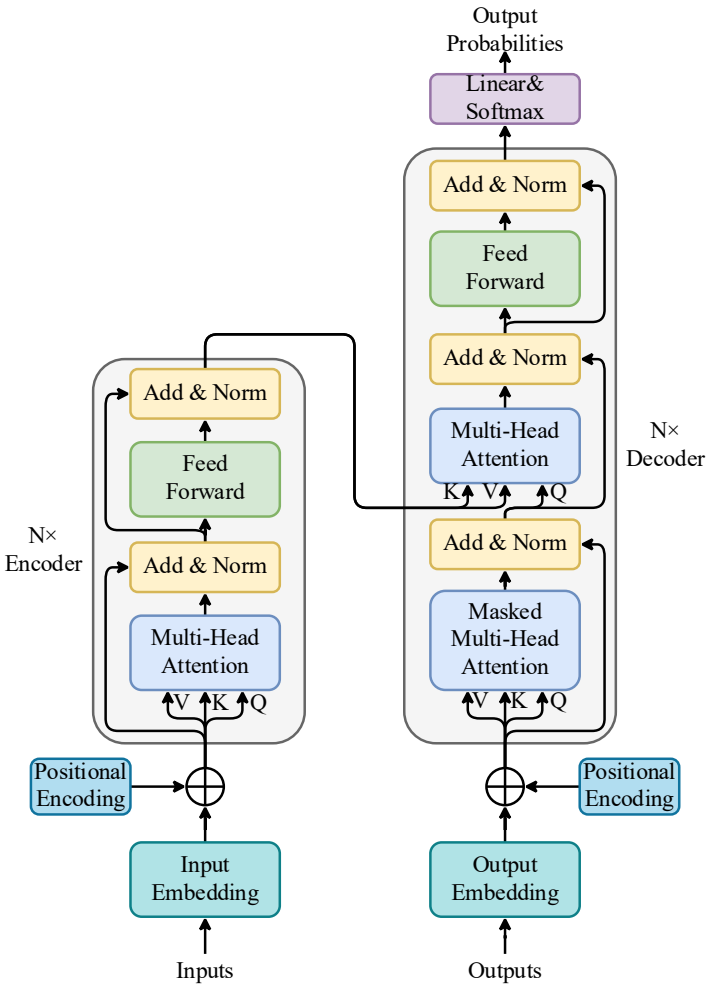


图 2.1 Transformer 整体结构概览[1]

词向量信息嵌入（Embedding）。模型的输入首先经过一个词向量嵌入层，将 one-hot 编码转换为维度为 d 的向量表示，该过程通常使用预训练词向量或构建可训练的随机向量。随后，将得到的嵌入向量与位置编码进行相加，送入多头自注意力模块中。位置编码通常采用正弦波输入的形式或者是可训练的向量嵌入。经过向量嵌入后，原始数据就对应到向量形式的抽象表示。原始数据在自然语言处理中可以是一个单词、一个汉字，在计算机视觉场景中可以是一组像素块，后文将原始数据对应的向量表示统一称作 Token。

多头注意力（Multi-Head Attention）。Transformer 模型架构的核心组件是多头注意力模块，该模块的主要思想是通过注意力计算，使得序列的每个 Token 可以从其他位置的 Token 中获取信息。对于模型的输入X，首先会通过线性映射得到多头空间的三个特征矩阵 Q（Query）、K（Key）和 V（Value）。根据注意力计算中 Query 和 Key-Value 对的来源，可以划分三种类型的注意力：

（1）自注意力。在编码器中，Q、K、V 的初始来源都是输入序列 X，X 为上一层输出，在第一层为初始模型输入。

（2）掩蔽自注意力。在解码器中，注意力的关注范围受到了限制，通过叠加一个掩蔽矩阵到注意力分数矩阵上，使得输入序列每个位置的 Query 只能关注到在它之前（包括自身）的 Key-Value 对。具体的，对于索引为 i 的 Query 和索引为 j 的 Key-Value 对，当$i < j$时，注意力分数$A_{ij} = -\infty$。

（3）交叉注意力。Query 由解码器的上一层输出提供，而 Key-Value 对由编码器的最终输出经过映射得到。

然后对 Q、K、V 特征矩阵进行如下的缩放点积注意力计算：

$$Attention(Q,K,V) = Softmax\left(\frac{QK^T}{\sqrt{d}}\right)V = AV \qquad (2-1)$$

$$Q,K,V = XW^Q, XW^K, XW^V \qquad (2-2)$$

其中$X \in R^{L\times d}$是模型的输入；$Q,K,V \in R^{L\times d_q}, R^{L\times d_k}, R^{L\times d_v}$是将输入序列映射到多头子空间后的向量表示；$W^Q, W^K, W^V \in R^{d\times d_q}, R^{d\times d_k}, R^{d\times d_v}$是线性映射的权重参数；$L$是输入序列的长度；$d$是原始输入特征向量的维度；$d_q$、$d_k$、$d_v$是多头注意力计算中的特征维度；$A$通常称为注意力矩阵，表示了$Q$和$K$中一对 Token 的相关程度；Softmax 操作以行的维度计算；Q和K的点积除以比例因子$\sqrt{d}$以缓解函数的梯度消失问题。

与简单地应用单个注意力函数不同，Transformer 使用多头注意力将原始维度d映射到 Q、K、V 对应的d_q、d_k、d_v，并分为 H 个不同的头空间。每个头空间独立地进行注意力计算，然后将所有子空间的输出拼接到一起，再通过线性映射恢复到维度为d的原始特征空间，如图 2.2 所示。

$$MultiHeadAttention(Q,K,V) = Concat(head_1, head_2, \ldots, head_H)W \qquad (2-3)$$

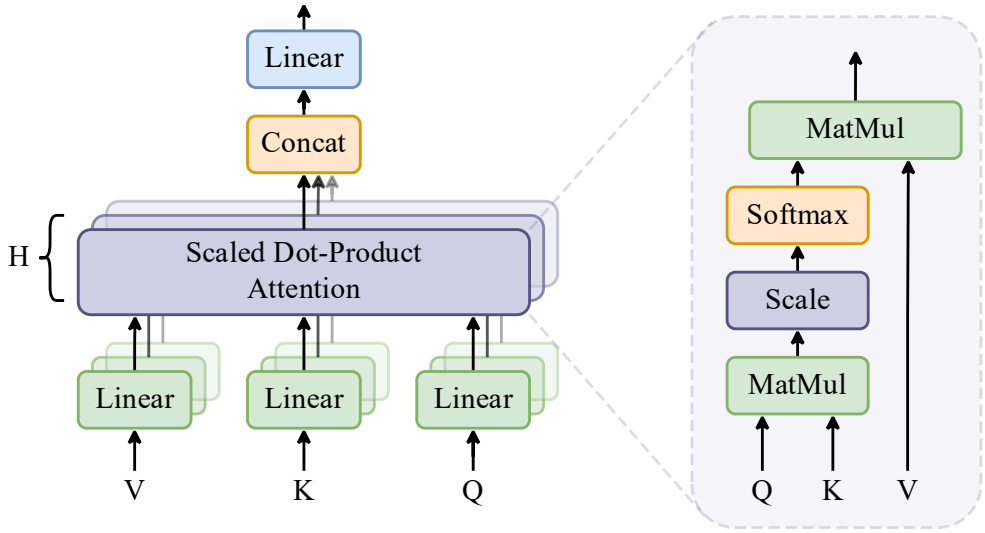


图 2.2 多头点积注意力计算过程[1]

前馈神经网络层（Feed Forward Layer）。注意力模块的输出被传递到带有ReLU 激活单元的两层前馈网络中，每个前馈层独立作用于每个位置，计算表示为：

$$FFN(X) = ReLU(XW^1 + b^1)W^2 + b^2 \quad (2-4)$$

其中，X 是多头注意力模块的输出，$W^1 \in R^{d_m \times d_f}$，$W^2 \in R^{d_f \times d_m}$，$b^1 \in R^{d_f}$，$b^2 \in R^{d_m}$都是映射函数的可学习参数。通常情况下作为中间向量的d_f的维度要比d_m大。

残差连接。为了构建深度模型，Transformer 使用了残差连接和归一化处理。多头自注意力模块的输入和输出通过残差层和归一化层进行连接，得到的输出作为输入被传递到两层的前馈网络中。前馈层的输入和输出同样经过残差连接和归一化层处理得到模块的最终输出。该部分的计算公式可描述如下：

$$X^h = LayerNorm(MultiHeadAttention(X) + X) \quad (2-5)$$

$$X^O = LayerNorm(FFN(X^h) + X^h) \quad (2-6)$$

2.1.2 模型复杂度和参数量级

Transformer 模型的计算需求来源于多个方面，但主要还是由多头自注意力和前馈神经网络两个关键模块构成。多头自注意力部分的计算主要来源于QK^T矩阵乘法操作，该操作的时间复杂度和空间复杂度都是$O(L^2)$级别。前馈神经网络层的复杂度与序列长度呈线性关系，但其开销也是不容忽视的。

假设模型输入序列的长度为 L，每个位置隐层向量表示的维度为 d，前馈神经网络的中间层向量表示的维度为 $4d$。那么注意力部分的计算复杂度为$O(L^2d)$，参数量为 $4d^2$。前馈神经网络层的计算复杂度为$O(Ld^2)$，参数量为 $8d^2$。综合两个主要模块，模型整体的计算复杂度为$O(L^2d + Ld^2)$。

当输入序列的长度 L 较短时，隐层特征维度 d 主要影响了注意力和前馈神经网络的复杂度量级，此时模型的计算瓶颈在前馈神经网络层。然而，随着输入序列长度 L 的增长，L 逐渐主导了两个主要模块的复杂度，注意力计算成为了模型的计算瓶颈。除此之外，注意力模块的计算需要存储注意力矩阵$A \in R^{L \times L}$，这导致模型在一些序列长度过长的场景会受到硬件设备内存的限制。

2.2 模型通用优化技术

深度学习模型从设计到应用需要经过一个复杂的阶段。从前期设计到框架层的实现、再到模型底层设备运行，整个流程涉及到硬件、软件等多方面内容。在

上层视角中，模型通用优化技术主要分为训练部署的加速和框架层的优化。

2.2.1 模型训练部署加速

分布式并行训练。在深度学习中，随着模型复杂度的增加以及数据规模的增大，单台设备或单张训练卡已经无法快速地训练模型。分布式并行训练可以利用多个计算设备同时训练一个模型，是提高模型计算效率的一个重要手段。根据并行的切分维度可以分为数据并行和模型并行两种基本类型，除此之外还有混合并行、流水线并行等组合策略。数据并行指在多个设备上放置相同的模型拷贝，将数据集分成几个部分送到不同的设备上进行训练。模型并行是指将一个神经网络切分成若干子模块分别运行于不同训练设备上，这些训练设备也可能分布在不同机器上。分布式并行训练可以利用多个计算设备的能力加快模型训练和推理的速度，同时也具有良好的扩展性，可以根据数据规模或模型的增大及时扩增设备数量，提高训练效率。两种基本并行策略的简单结构如图 2.3 所示，左图为数据并行示例，右图为模型并行示例，每个虚线框代表一个计算设备。

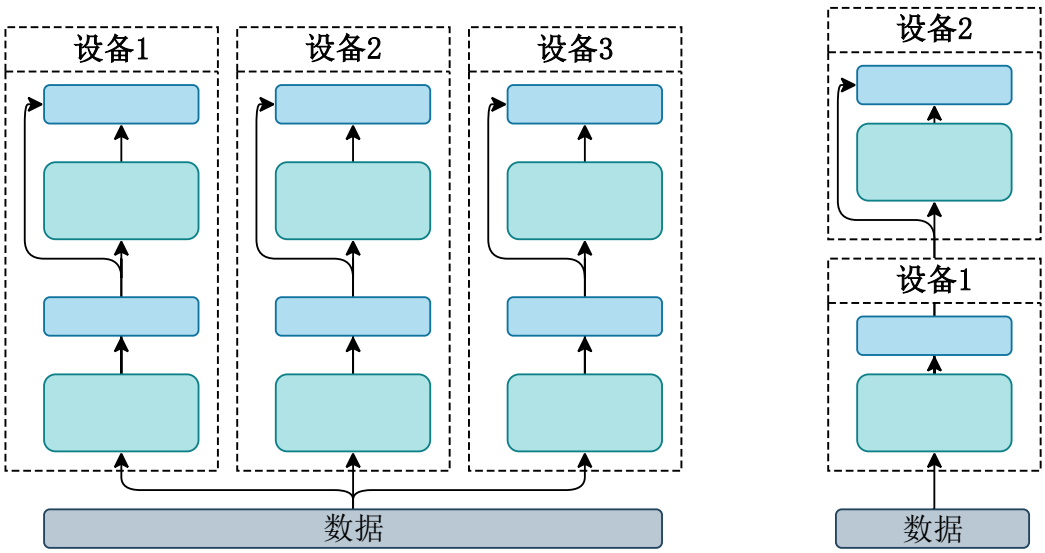


图 2.3 数据并行（左）和模型并行（右）示意图

定制化加速器。Transformer 模型的核心计算是矩阵乘法，对核心计算进行优化也是模型加速的重要手段之一。通过对模型的热点运算单元进行定制化软硬件协同设计[31]，可以在 GPU、TPU 和 FPGA 等硬件加速器上对计算进行深度优化，加速模型的计算过程。

自适应优化算法。传统的优化算法通常需要手动选择学习率和其他参数，自适应优化算法可以根据模型和数据的特征自动调整学习率和其他参数，以提高训练效率。常用的自适应优化算法包括 Adagrad、Adadelta、Adam 等。

2.2.2 框架层优化

计算图优化。深度学习模型在框架层会被抽象成由节点和边构成的计算图，节点表示运算单元或变量，边表示数据的流动。通过对计算图进行等价变换、算子融合、常量折叠和图剪枝等操作[32]，可以减少计算图的复杂度，使其更适合特定的硬件和场景，最终提高执行效率。当前主流的深度学习框架如 TensorFlow、PyTorch 等都提供了计算图优化技术的支持。

自动混合精度。混合精度技术是在模型参数的保存与运算过程中混合使用不同精度参数来加速训练和推理的技术[25]。该技术主要使用了两种数据类型：单精度浮点数（32 位）和半精度浮点数（16 位）。使用半精度浮点数或更低精度浮点数可以减少计算量、内存占用和内存带宽等，因此具有较高的计算效率。然而半精度浮点数容易产生数值精度问题，导致梯度消失或溢出，影响网络训练和推理的效果。混合精度技术使用多种精度进行计算，在适当的地方使用半精度浮点数，在保证模型准确性的同时提高模型计算效率。主流硬件和软件平台（如 NVIDIA 和 TensorFlow）也已经支持了自动混合精度技术。

2.3 Transformer 模型优化加速技术

Transformer 作为如今许多领域模型的基础，有许多针对其结构的特定优化技术。本节主要从模块级、Token 级和参数级三个方面进行介绍。

2.3.1 针对注意力模块的稀疏优化

注意力模块是 Transformer 模型的核心组成部分之一，也是模型计算消耗的主要来源。因此，为了提高 Transformer 模型的计算效率，发展出了很多针对注意力模块的优化技术。其中，稀疏注意力是近些年的一种代表性技术。稀疏注意力机制[14][15][16]使用局部注意力来代替全局完整的注意力机制，减少了计算量和内存占用。

在标准的注意力机制计算过程中，每个 Token 都需要和其他所有位置的 Token 进行注意力计算。然而注意力矩阵往往是非常稀疏的，在很多位置上的值非常小，进而在信息交互过程中的贡献也非常微弱。因此，可以通过引入结构偏差来限制每个 Query 所能关注的 Key-Value 对的数量，进而降低计算复杂度，这也就是稀疏注意力的核心机制。

$$A_{ij} = \begin{cases} q_i k_j^T & \text{如果} i \text{和} j \text{是可见的} \\ -\infty & \text{如果} i \text{和} j \text{是不可见} \end{cases} \tag{2-7}$$

从另一方面来说，标准注意力可以看作是一个完全二部图，其中每个 Query 从所有节点接收信息并更新其隐层表示。稀疏注意力可以被认为是一个稀疏图，其中部分节点对之间的连接被移除。图 2.4 展示了两种不同注意力矩阵的差别，左图标准注意力矩阵中每两个节点所对应的位置都存在非零值，右图空白无色位置表明对应的两个节点间未计算注意力分数或注意力分数为零。

图 2.4 标准注意力和稀疏注意力的权重矩阵对比

按照稀疏模式的参数评估策略，可以简单分为基于位置的稀疏模式和基于内容的稀疏模式两类。

基于位置的稀疏注意力。基于位置的稀疏注意力根据一些预定义的模式来限制注意力的视野范围。这些稀疏模式的种类很多，但可以由一些基本的稀疏模式组合而成。

（1）全局注意力。局部稀疏注意力会导致模型长距离依赖关系建模能力的下降，为了缓解该问题，可以引入一些全局节点作为枢纽。这些全局节点可以与序列中的所有节点进行注意力信息交互，其他所有位置的结点也都可以注意到全局结点。

（2）带状注意力。对于大多数数据来说，原始数据转换为 Token 序列之后都表现出很强的局部相关性。因此可以将每个 Token 所能关注到的范围限制在一定距离内的邻居节点上。这种稀疏模式可以称之为带状或滑动窗口注意力。

（3）间隔注意力。如果限定 Token 所能关注到其他位置 Token 的数量，那么可以通过增加两个互相可见 Token 之间的间隔，使得 Token 可以关注到更远距离的 Token，将局部密集型视野范围转换为广度稀疏型。

（4）随机注意力。为了提升非局部的信息获取能力，对于每个 Query 随机选取若干个可以关注到的 Key-Value 对构成一个随机关联图。信息可以在该图上通过随机游走的形式快速地进行传递交互。

（5）块状注意力。该类注意力将输入序列分割成若干个互不重叠的 Query 块，每个块与一个局部相邻块相关联。一个 Query 块中的所有 Query 只关注对应局部块中的 Key-Value 对。

基于位置的稀疏注意力或单独使用一种稀疏模式，或采用多种模式的混合。然而不论何种策略都是与输入数据无关的固定视野范围模式，是基于人的经验而设计的先验偏差方案。在具体实现上，部分稀疏模式通过掩蔽矩阵达到控制可见范围的目的，这在模型效率上几乎没有提高。还有一些稀疏计算的高效执行需要定制化计算单元和硬件设备的适配支持。

基于内容的稀疏注意力。基于内容的稀疏注意力同样是为了提高注意力机制效率的计算策略。这种机制的基本思想是基于输入内容创建稀疏图，即稀疏连接是基于输入样本动态构建的。构建基于内容的稀疏图的一个简单方法是利用输入序列的内容信息，如语义或句法等，来指导序列中 Token 相关度的计算。实际应用中，一般需要使用基于大量数据训练得到的预训练词向量（例如：Word2Vec[33]、Glove[34]等），以此更准确地计算不同位置向量的相关度。基于向量相关度从输入序列选择一个子集而不是所有，并将此作为 Token 的视野范围，同时得到一个稀疏连接图。对于存在连接关系的两个 Token 节点计算它们的注意力分数，由此得到一个稀疏的注意力分布。为了有效地构建稀疏图，可以参考最大内积搜索(MIPS)问题：在不计算所有点积项的情况下，尝试寻找与指定 Query 具有最大点积的 Key。

基于内容的稀疏注意力机制能够保留更多的语义信息，并且能够更精确地注意到有意义的内容，提高了注意力的效率。然而其效果依赖于稀疏图构建的准确性，并且在 Token 间存在不可避免的视野盲区，这会导致信息交互的损失。

本文第三章的工作借鉴了基于内容的稀疏注意力并将其作为核心思想，同时尝试对该策略存在的问题进行解决。

2.3.2 针对 Token 的选择优化

基于 Transformer 架构的模型在多个领域都有深度应用。在自然语言处理场景中，模型输入一般是文本序列。该序列的长度从几个词到几百个词不等，甚至在有些长文本文档场景中达到了上千词。在计算机视觉领域中，模型输入一般是对图片像素点进行采样后的序列。该序列的长度随着图片分辨率和采样块的大小变化也随之有较大变化，同样存在长序列的问题。是否整个输入序列中的每一个 Token 都对模型的输出具有绝对的重要性呢？如果不是，哪些更重要呢？针对这个问题，需要设计评估策略对序列中不同 Token 的重要性进行量化。目前有如下几种常见的方法：

（1）基于梯度的策略[35]：通过计算梯度值来评估 Token 的重要性。具体来说，将模型预测函数的输出作为结果，对 Token 序列中的每一个元素进行求导得到对应的梯度值。梯度值越大，说明该 Token 对于预测结果的贡献就越大，进而可以认为该 Token 的重要性就越强。该过程通常可以用类似如下的公式进行计算。

$$S_i = \left\| \frac{\partial y_c}{\partial h_i^0} \odot h_i^0 \right\|_2 \tag{2-8}$$

（2）基于注意力的策略[36][37]：具体而言，在多头自注意力模块中，输入序列的每个 Token 会和其他 Token 计算特征向量的内积，得到注意力分数矩阵。该矩阵中的权重值决定了 Token 之间信息影响的程度。基于注意力权重可从不同维度进行计算，得到 Token 的重要性评分。依据重要性评分对输入序列的所有 Token 进行排序，可以得到一个重要性评分排名。总的来说，这是一种高效且可靠的 Token 重要性评估方法，其核心原理的计算过程通常如下所示。

$$Score = \frac{QK^T}{\sqrt{d}} \tag{2-9}$$

（3）可学习的动态策略[38][39]。该类型策略通常用带有可学习参数的结构对 Token 的重要性进行评估，将该过程作为一个子任务参与到模型训练过程中同步进行学习。比如使用简单的线性打分函数：$S(X) = XW^T + b$，W和b都是可训练参数。或者使用强化学习等思想对 Token 的选择方案进行训练。该策略是一个动态学习的过程，往往需要适当的评估方法对训练中参数的学习效果进行衡量。

本文第 4 章使用了基于注意力方法的评估策略，基于不同对象间的注意力矩阵设计了若干策略对 Token 的贡献度进行衡量。基于贡献度对 Token 进行选择，用以提高模型效果和模型效率。

2.3.3 针对模型参数的剪枝优化

模型剪枝是一种减少 Transformer 模型参数量和计算量的有效方法[22][23][24]。该方法的基本思路是对模型中冗余的参数和层进行剪枝和压缩，以减少计算和存储的成本。模型的修剪方法大致可以分为两类：一种是非结构化的修剪方法，通过将参数绝对值或梯度和预先定义的阈值进行比较来修剪单个权重，低于阈值的被置为 0。第二种是结构化剪枝，旨在修剪一个完整结构的权重值，甚至是模型中完整的结构组件。例如，可以使用低秩分解、剪枝、量化和哈希等技术对权重矩阵进行压缩，或者对 Transformer 的某些层进行裁剪。

2.4 本章小结

本章主要介绍了与本文相关的各方向技术理论。首先介绍了 Transformer 的模型架构，并对其主要模块和使用流程进行了介绍，同时对其复杂度和参数量级进行了分析。然后，从训练部署方案和训练框架两个层面介绍了模型效率优化的一些通用技术。最后，针对 Transformer 模型的优化加速技术，从模块级、Token 级和参数级三个方面介绍了对应的具体优化策略。

第 3 章 基于层内分组的稀疏注意力策略

3.1 研究动机与方法概述

Transformer 架构模型的注意力矩阵呈现出了典型的稀疏分布特征。在注意力矩阵中，权重值越大，说明对应位置的两个 Token 之间的相关性越强，它们之间通过注意力模块所交换的信息也就越多。因此，可以将分组稀疏模式应用到注意力矩阵的计算上，以此把有限的计算资源投入到核心的关键节点上，提高模型效率。然而，分组稀疏注意力的有效性依赖于分组的准确性。同时，在面向分类任务时，序列在经过分组后难以通过单一 Token 获取整个序列的综合信息。

针对如上问题，本章在一层编码器内部横向地对输入序列的 Token 进行聚类分组，将潜在更为相关的 Token 划分到同一个分组中，同组的两个 Token 之间的注意力分数应该是大于不同组 Token 间的注意力分数的。在各组组内进行注意力机制计算，使重要的信息在组内流动交互。为了提高分组的准确性，在分组流程的前后期分别提出不同的策略进行综合。在分组前期，提出两种基于输入序列的聚类中心初始化策略，并在根据注意力分数进行分组时综合考虑预训练词向量和距离等信息。在分组后期，提出聚类中心信息增强策略和分组损失计算方法，以此对基于注意力相关性的分组过程进行修正。

针对分组注意力在分类任务上的应用，设计一种信息层次融合的方法，逐层级地将各 Token 的信息汇聚到全局向量中，以更高效地获取整个序列的综合信息。

通过上述多种优化策略，以尽可能地减少分组过程产生的误差，提高分组的合理性与准确性，在提高模型效率的同时，尽可能保证甚至提高模型效果。本章的整体模型结构如图 3.1 所示，绿色虚线部分为初始化流程，只需执行一次，完整处理流程分为如下五个主要阶段。

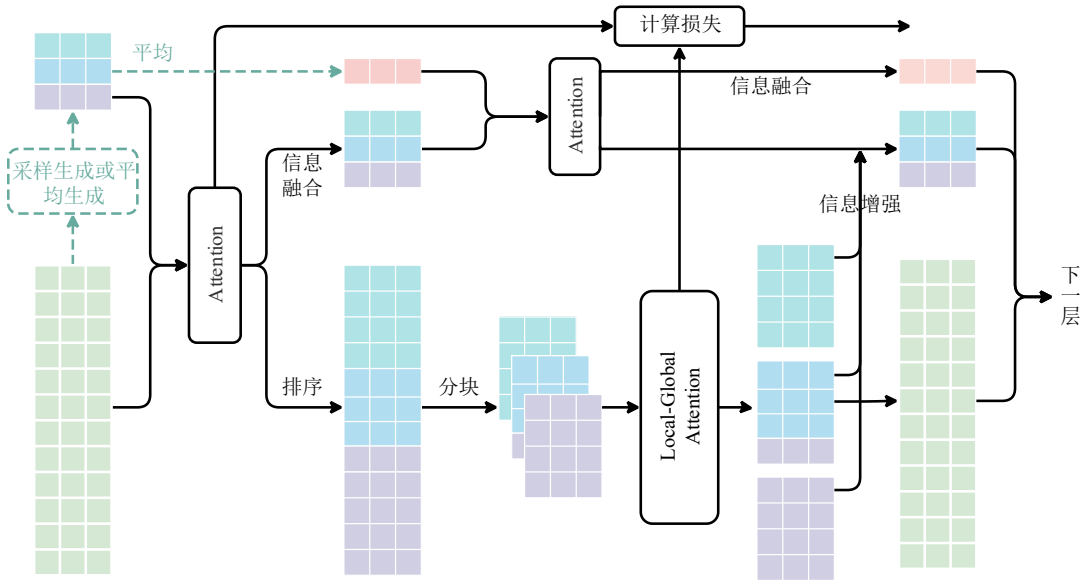


图 3.1 基于层内分组的稀疏注意力模型整体结构

3.2 Token 分组注意力流程

3.2.1 分组视野范围

在基于聚类分组的注意力计算过程中，长度为L的输入序列需要被分成多个不同的片段，以此构成不同的分组。为了能更好地利用硬件的并行计算能力，按照分组索引排序后的序列会被裁剪为长度统一的多个分组片段。经过上述处理之后，标准的注意力机制计算独立地在各组内部进行，因而每组所包含 Token 的数量L_c就关键影响了组内注意力中各个 Token 所能关注到的范围，后续简称为“视野范围”。

本文对输入序列进行聚类分组处理的初衷是通过分组将注意力的视野范围限制在语义关联度相对更强的序列片段中，每个组的大小决定了该组的视野范围。基于现实世界对于语言的常识认知，视野范围应该是一个基于大规模语料统计后的范围值，而不应该随输入序列长度和分组的数量产生大范围波动。因此，在后续的工作中，更倾向于优先确定每个分组的序列长度L_c，即视野范围。在这之后，将序列填充到视野范围L_c的整数倍，再动态计算得到聚类分组的数量。对于视野范围的合理取值问题，本文在后续实验章节进行相关参数实验，通过实验结果确定较为推荐的视野范围值。

3.2.2 聚类中心向量的生成

在基于注意力对输入序列进行分组时，需要先确定N_c个聚类分组中心$C \in$

$R^{N_c \times d}$。然后，将聚类中心C与输入序列的嵌入表示X进行注意力计算，C和X分别作为 Query 和 Key-Value 的信息来源。该过程一方面可以使聚类中心从输入序列中获取信息，另一方面也可以得到两者之间的注意力分数矩阵。通过注意力分数矩阵可以判定输入序列中的 Token 和哪一个聚类中心更为相关，以此将 Token 划分到对应的聚类分组中。

在模型的第一层处理过程中，聚类中心向量缺少来自于前一层的传递信息，因此需要进行初始化生成。该过程生成的聚类中心会对第一层分类的准确性产生较大影响，它带来的误差会因为第一层的位置特性，最终扩散到模型的整个后续流程，影响模型效果。因此，如何更合理地生成初始聚类中心向量是一个非常关键的问题。针对该问题，本章提出两种基于输入序列构造初始聚类中心向量的策略，而摒弃了 ClusterFormer 中的初始化方案。通过使用新的生成策略，以期能获得更准确的初始向量表示，进而提高整个分组过程的准确性，最终提高模型效果。

基于距离的平均生成策略。聚类中心向量某种意义上可以视作一部分 Token 序列的综合表示，所以其初始生成也可以考虑对输入序列进行综合。在对语言理解的日常认知中，物理距离上相距更近的词之间更倾向于在描述同一件事情，其语义相关度更大。该策略将这种距离信息引入到聚类中心的初始化生成过程中，对于模型的输入序列X，基于原始排列顺序对其进行均等分段，然后计算每个子序列向量表示的平均值，由此得到初始的聚类中心向量，详细过程如算法 1 所示。

算法 1 基于距离的平均生成策略

输入：样本序列$\boldsymbol{X} = [w_1, w_2, \dots, w_L]$

输出：聚类中心$\boldsymbol{C} = [c_1, c_2, \dots, c_{N_c}]$

1：$L_c = L \,//\, N_c$

2：**for** $i = 0; i < N_c; i++$ **do**

3：　　$C[i] = mean(w_{i \times L_c}, \cdots, w_{(i+1) \times L_c})$

4：**end for**

其中，L为输入序列的长度；N_c为聚类中心的数量；L_c为分组片段的统一长度。

基于距离的平均生成策略，其主要计算集中在求片段序列的平均向量过程。因为批处理中每一个样本的计算逻辑都是相同的，所以可以利用硬件（如 GPU）的并行化计算能力加快计算速度。除此之外，该策略生成聚类中心时将距离信息隐式地加入到了向量表示中：每个聚类中心的向量表示是在一定物理距离范围内的序列向量的平均值，一定程度上跟日常语言认知是统一的。通过该策略生成的聚类中心基于样本获取相关信息，对信息进行综合时考虑了距离因素，如此获得的向量表示理论上更为合理，可以提高后续分组过程的空间合理性。

基于 K-Means++的采样生成策略。聚类中心的向量表示在分组阶段会和输入序列的向量表示进行注意力计算，也就是认为两者在计算过程中是属于同一个特征空间的。基于该假设，提出采样生成策略从输入序列中采样选取样本，并将其作为初始聚类中心向量。从长度为L的序列中选取N_c个样本作为聚类中心，然后基于聚类中心对输入序列进行分类，该过程类似于 K-Means 算法的处理逻辑。考虑到对输入序列进行分组处理的目的之一是尝试将相关度更强的 Token 划分到同一组，同时对不同组的表示空间尽可能进行区分。在理想状态下，归属于同一个分组的 Token 在抽象信息层面是具有强相互关联关系的，而归属于不同组的 Token 之间关联关系要较为弱一些。因此，本章选择参考 K-Means++算法的处理流程对采样策略进行优化，期望以此达到组内高内聚、组间低耦合的 Token 分布状态。具体的，基于 K-Means++算法采样生成初始聚类中心的整体过程如算法 2 所示。

算法 2 基于 K-Means++的采样生成策略

输入：样本序列$\boldsymbol{X} = [w_1, w_2, \dots, w_L]$

输出：聚类中心$\boldsymbol{C} = [c_1, c_2, \dots, c_{N_c}]$

while $\text{len(C)} < N_c$ **do**
  if $\text{len(C)} = 0$ **then**
    随机获得第一个样本的下标$0 \le i \le L$；
    获得第一个聚类中心$c_1 = w_i \in X$；
    $C = [c_1]$；
  else:
    for $n = 0;\ n < L;\ n++$ **do**
      $d_n = 0$；
      for $m = 0;\ m < len(C); m++$ **do**
        $d_{mn} = distance(w_n, c_m)$；
        $d_n = d_n + d_{mn}$；
      end for
    end for
    对距离d进行正则化
    聚类中心选择概率为$P = \frac{d_i}{\sum d_j}$；
    依据 P 的概率分布选出下标j；
    $C[len(C)] = w_j$；
  end if
end while

基于 K-Means++的采样生成策略在选择初始聚类中心时考虑了输入序列与已选择的聚类中心向量的关联度，此处关联度的计算与分组过程的计算方法保持一致，即注意力机制中的向量内积操作。序列中的 Token 与已选择聚类中心的关联度越高，就越可能属于所属类别，因而被选择成为下一个聚类中心的概率也就越小。如图 3.2 所示，绿色代表已选择的聚类中心，其余节点颜色越深被选择的可能性越大，绿色框对应的 Token 是当前选择轮次中被选择概率较大的节点。通过上述策略的处理，从样本中选取到的聚类中心相互之间的关联关系尽可能地弱，也就区分了不同组的表示范围，达到组内高内聚、组间低耦合的理想状态。

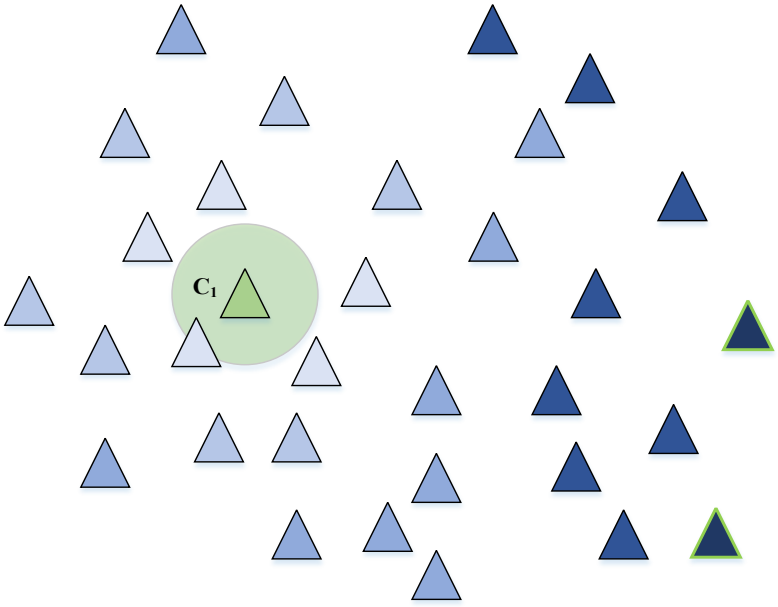


图 3.2 采样生成聚类中心示意图

3.2.3 基于注意力的分组计算

在对输入序列进行聚类分组的过程中，首先要获取聚类中心向量$C \in R^{N_c \times d}$。将输入序列的隐层表示向量$X \in R^{L \times d}$和聚类中心向量C进行注意力计算，C作为 Query 的来源，X作为 Key 和 Value 的来源，由此得到融合了序列信息的聚类中心上下文C^I，以及序列 Token 与聚类中心向量的注意力矩阵$ScoreCX$。

$$ScoreCX = \frac{C_i W^Q \cdot \left(X_j W^K\right)^T}{\sqrt{d}} \qquad (3-1)$$

$$C^I = Softmax(ScoreCX) \cdot X_j W^V \tag{3-2}$$

$$1 \le i \le N_c, 1 \le j \le L$$

其中，N_c为聚类中心的数量；L为输入序列的长度；$W^Q, W^K, W^V \in R^{d \times d_{head}}$是输入序列映射到多头隐层特征空间的参数矩阵。$ScoreCX_{i,j}$代表的是第$i$个聚类分组中心$C_i$和第$j$个输入序列$X_j$对应的特征向量进行内积计算得到的值，反应了$C_i$和$X_j$的注意力相关度。$C^I$作为从输入序列获取的上下文信息，将通过残差和归一化处理融入原始的聚类中心向量中。

依据语言理解的日常认知，距离上相距更近的 Token 之间内容关联度也会更强。因此，本章基于输入序列与聚类中心的距离信息生成一个比例缩放矩阵，显式地将距离信息叠加到输入序列X和聚类中心C的注意力矩阵上，进而使得距离上相距更近的 Token 更容易被分到同一个分组片段中。

$$DistanceMask = 1 - \frac{|i-m|}{N_c * \alpha} \tag{3-3}$$

$$1 \le i, m \le N_c$$

其中，i对应聚类中心的索引，m对应 Token 所在分组片段的索引；α作为比例因子，用以控制距离信息的影响程度。考虑到注意力分数存在负值，因此当原本的注意力分数为正值时直接与缩放因子相乘；如果注意力分数是负值，则与缩放因子的倒数相乘。具体的，对于聚类中心和输入序列的注意力矩阵$S = ScoreCX$，进行如下的放缩变换：

$$ScoreCX^D = \begin{cases} S_{i,j} * DistanceMask_{i,\left[\frac{j}{L_c}\right]} & if\ S_{i,j} \ge 0 \\ \dfrac{S_{i,j}}{DistanceMask_{i,\left[\frac{j}{L_c}\right]}} & if\ S_{i,j} < 0 \end{cases} \tag{3-4}$$

其中，$S_{i,j}$代表聚类中心和输入序列的注意力矩阵，i为聚类中心索引，j为输入序列的索引，$[j/L_c]$为输入序列除以分组片段长度后的取整值，对应原始排列中的分组索引。基于加入距离信息后的注意力矩阵$ScoreCX^D$，可以获取与X_j注意力分数最大的聚类分组中心C_i及其对应的分组索引号。按照分组索引号对输入序列进行排序，即可得到同组相邻排列的 Token 序列。

$$I_j = Argmax\left(ScoreCX^D_{:,j}\right) \quad 1 \le j \le L \tag{3-5}$$

属于同一个聚类分组的 Token 的数量可能并不是完全统一相等的，这就使得难以利用硬件的并行计算能力来执行不同组的注意力机制计算。针对这种情况，

需要对输入序列进行非严格分组：获取输入序列各个 Token 所属的分组索引后，按照索引进行排序，排序之后再将一维输入序列平均展开为大小相等的二维序列，保证每个分组片段包含相等数量的 Token。虽然同一个片段内的 Token 对应的分组索引可能不一致，但也将其视为属于同一个聚类分组，如此能获得更高的计算效率。

3.2.4 组内注意力计算

对输入序列的向量表示$X \in R^{L\times d}$进行聚类分组处理之后，得到按照分组类别排序的输入序列X^S。随后的流程与标准 Transformer 的处理一致，首先通过可学习的线性映射将输入序列映射到多头空间。

$$Q^S, K^S, V^S = X^S W^Q, X^S W^K, X^S W^V \tag{3-6}$$

其中，$W^Q, W^K, W^V \in R^{d\times d_{head}}$是对应的权重矩阵，$Q^S, K^S, V^S$分别对应自注意力模块的 Query、Key 和 Value。然后按照预先确定好的组大小N_c将排序后的序列分成大小相同的组。

$$(Q^{C_i}, K^{C_i}, V^{C_i}) = (Q^S, K^S, V^S)_{(i-1)L_c,\cdots,iL_c} \tag{3-7}$$

$$1 \le i \le N_c$$

其中N_c是聚类分组的大小，满足$N_c \times L_c = L$。

通过分组处理，每个组内的 Token 只与同组 Token 互相可见，各组的组内注意力计算就可以并行进行，互不影响。

$$H^{C_i} = Attention(Q^{C_i}, K^{C_i}, V^{C_i}) \tag{3-8}$$

在各组进行注意力计算之后，将输出拼接到一起，然后通过分组排序过程的索引恢复到分组前的顺序，得到注意力的上下文表示，再从多头空间映射回原始特征空间。通过残差形式将输入与上下文相加并进行归一化，得到注意力模块的输出X^O。

$$H^C = Concat(H^{C_1}, H^{C_2}, \dots, H^{C_{N_c}}) \tag{3-9}$$

$$X^C = ReverseSort(H^C) \tag{3-10}$$

$$X^O = LayerNorm(X + X^C W^O) \tag{3-11}$$

3.2.5 注意力分组的修正

在对输入序列进行聚类分组处理时，本文以输入序列与聚类中心向量的注意

力分数大小作为依据，来获取与 Token 关联度最大的聚类中心的索引。在分组注意力计算过程中，同属于一个分组的 Token 之间的注意力分数被期望相比于不同组间的注意力分数要更大，这是分组注意力机制有效性的前提。然而，由于实际计算过程中并没有计算任意两个 Token 之间的注意力分数，而是通过 Token 和聚类中心向量的注意力分数进行了粗略划分，这就导致了这种基于传递相关性的分组方案存在一定的不准确性。为了对该方案产生的误差进行修正，本章提出聚类中心信息增强策略和分组损失计算策略。

聚类中心信息增强策略。在基于注意力计算进行分组时，聚类中心根据和 Token 的相关性从整个输入序列获取信息，得到整体的上下文C^I。在此之后，属于同一个分组的 Token 通过组内注意力计算进行信息交互，获取组内上下文信息。该维度的信息带有分组局部性特征，将该信息跟聚类中心的向量表示进行融合，便可以对聚类中心进行组内信息增强。具体实现上，通过组内注意力计算可以得到各分组片段的上下文信息表示$H^{C_i} \in R^{L_c \times d}$，其中$L_c$为分组片段的长度，$d$为向量的隐层表示维度。对于该信息表示，在$L_c$的维度上进行平均计算操作，得到每个分组内部 Token 的平均信息$C^M \in R^{N_c \times d}$。

$$C^M = \sum_{j}^{L_c} \frac{H_j^{C_i}}{L_c} \tag{3-12}$$

其中，C_i^M代表了属于第 i 个聚类分组的 Token 的平均信息表示。将此信息叠加到聚类中心的向量表示上并进行归一化处理，实现对聚类中心的信息增强，提高后续分组过程的准确性。

$$\hat{C} = LayerNorm(C + C^I + C^M) \tag{3-13}$$

其中，$\hat{C} \in R^{N_c \times d}$是融合了输入序列信息和增强信息后的聚类中心向量表示。

分组损失计算策略。针对基于传递相关性的注意力分组过程，本章提出对应的误差计算策略，以此将分组过程当做一个子任务加入到模型训练中。自注意力机制的核心原理是根据两个 Token 的内积来评估其关联程度。在本文的前置处理中，进行了传递相关性的假设：如果两个 Token 都与同一个聚类中心向量的注意力分数更大，那么这两个 Token 之间的注意力分数也应该尽可能大。进而根据 Token 与聚类中心的注意力分数对输入序列进行了分组。

在一个分组内部，每个 Token 都会和对应的聚类中心向量通过内积计算得到一个注意力分数$ScoreCX \in R^{N_c \times L_c}$，该分数衡量了 Token 与对应的聚类中心之间的相关度强弱。对于组内的各个 Token 之间，会在组内注意力阶段通过内积计算得到两两之间的注意力分数$ScoreXX \in R^{N_c \times L_c \times L_c}$，该分数衡量的是组内 Token 之

间的相关度强弱。注意力分组的过程，其实是基于 Token 和聚类中心的关系来预估 Token 与 Token 之间关系的过程，因此这两种关联关系的分布应该尽可能趋于一致。具体的，对于一个分组，将组内所有 Token 与该组聚类中心的注意力分数进行平均作为聚类中心和组内 Token 的关系分布，将同组所有 Token 相互之间的注意力分数进行平均作为组内 Token 间的关系分布。两个关系分布对应的差值作为注意力分组的损失，将该值作为额外的损失来源加入到模型的损失中，约束二者的注意力分布使其尽可能一致，以此对注意力分组方法进行修正。

$$DistriCX_i = \frac{\sum_j ScoreCX_{i,j}}{L_c} \tag{3-14}$$

$$DistriXX_i = \sum_j \sum_k \frac{ScoreXX_{i,j,k}}{L_c * L_c} \tag{3-15}$$

$$Loss_c = \frac{\sum_i |DistriCX_i - DistriXX_i|}{N_c} \tag{3-16}$$

其中，$DistriCX_i$代表第 i 个分组片段中聚类中心和组内 Token 的注意力分布；$DistriXX_i$代表第 i 个分组片段中 Token 之间的注意力分数分布；$Loss_c$代表两个分布之间的误差，最终加入到模型误差中进行协同训练。

3.3 基于分类信息层次融合的注意力结构扩充

在面向分类任务的处理过程中，基于 Transformer 的主流模型方案会在输入序列的头部添加一个特殊的分类标识向量[CLS]，该特殊分类向量会作为整个序列的一个特殊 Token 存在。输入序列在模型中通过注意力机制进行信息交互时，[CLS]也会和其他所有位置的输入 Token 进行交互以获取全局信息。通过此种方式，[CLS]就能够获取整个输入序列的抽象分类信息，在模型最后一层编码器输出后通过一个线性映射层将d维的特征向量映射到样本类别数量C的维度，再通过 Softmax 归一化处理后获取样本属于每个类别的概率，以此作为预测输出。通过将预测的类别与标注类别进行比较来计算误差，将误差进行反向传播使得模型参数学习到输入数据中的抽象分类特征。

在分组注意力过程中输入序列被划分到不同的组中，注意力的视野范围被限制在每个独立分组的内部。因此，常规分组处理中就缺少一个向量能和输入序列的所有位置进行注意力信息交互，以存储序列层面的整体信息。针对该问题，本章提出分类信息层次融合的方法来实现分类信息从单个 Token 到聚类中心再到全局分类向量的传递融合。

词级别到组级别的信息传递融合。在基于注意力的分组计算过程中，输入序

列的每个 Token 会与聚类中心向量进行注意力计算。在该过程中聚类中心向量可以从输入序列捕获到抽象分类信息，此为 Token 级别信息到聚类中心级别的传递融合。

$$C^I = Softmax\left(\frac{C_i W^Q \times \left(X_j W^K\right)^T}{\sqrt{d}}\right) \times X_j W^V \qquad (3-17)$$

其中，C_i为索引位置为i的聚类中心的向量表示，X_j为索引位置为j的输入序列 Token 的向量表示。

组级别到全局的信息传递融合。参考主流方案对分类任务的处理，本章额外设置了一个全局向量$Class \in R^d$作为整个序列抽象信息的代表，其初始值由各聚类中心向量取平均得到。在每层模型的处理过程中，全局向量$Class$首先使用与输入序列X相同的 Query 线性映射层将特征空间映射到多头子空间中。然后，全局向量与聚类中心向量的上下文信息C^I进行注意力计算，以此将信息从中心向量传递到全局向量中进行融合。

$$Class^I = Softmax\left(\frac{Class \cdot W^Q \times (C^I)^T}{\sqrt{d}}\right) \times C^I \qquad (3-18)$$

其中，$Class$为设置的全局向量，用以存储整个序列层面的信息。在注意力计算过程中得到的上下文信息与原全局向量表示通过残差层进行连接，再通过 LayerNorm 层进行归一化处理，得到全局分类向量在该层的输出。

$$Class^O = LayerNorm(Class + Class^I) \qquad (3-19)$$

随着模型在纵向层与层之间的逐层计算，最终在模型的输出层可以得到全局分类向量的表示，再通过一个线性分类器将向量从特征维度映射到类别维度，以此来进行分类任务。

3.4 复杂度分析

基于层内分组的稀疏注意力模型，其计算可以分为如下几个主要方面：

（1）聚类分组过程。在该过程中，将聚类中心向量$C \in R^{N_c \times d}$和输入序列的向量表示$X \in R^{L \times d}$进行注意力计算，该部分的复杂度为$O(LN_c d)$。

（2）组内注意力计算。经过聚类分组处理之后，长度为L的输入序列被分成了N_c个分组，每个分组内部的序列长度为L_c，满足$L = N_c \times L_c$。每个分组进行独立的注意力计算，该部分的复杂度为$O(N_c L_c^2 d) = O(LL_c d)$。

（3）信息传递融合。信息传递融合过程分为两个阶段，第一个阶段在分组

聚类过程完成，该部分计算已包含在分组聚类过程。第二阶段为聚类中心到全局级别的信息传递，该阶段将N_c个聚类分组中心和全局分类向量进行注意力机制计算，整体的复杂度为$O(N_c d)$。

综合上述处理过程，整个模型的复杂度为$O(LN_c d + LL_c d + N_c d)$，如果将每个聚类分组的大小设置为$L_c = \sqrt{L}$，则复杂度可以等价为$O(L\sqrt{L}d)$。

模型的参数主要来源于多头自注意力空间映射和前馈神经网络层，这两个模块的参数量主要取决于输入序列的长度。基于分组的注意力机制只是在注意力计算过程中进行了稀疏分组处理，在其他模块并未对序列进行改变。因此，本章模型与标准的 Transformer 具有一致的模型参数量，但在实际运行过程中，本章模型要存储的中间变量更少，占用的显存也更少。

3.5 本章小结

本章聚焦于 Transformer 模型多层堆叠编码器的单独一层内部，在横向层内的视角设计了基于分组的稀疏注意力计算。在分组流程的不同阶段提出了多种优化策略：在前期聚类中心向量生成阶段，提出了基于距离的平均生成策略和基于 K-Means++的采样生成策略，旨在提高分组的准确性。在分组过程的后期，设计了聚类中心信息增强策略和分组损失计算方法，旨在对基于聚类中心的注意力分组过程进行优化。多种策略综合，在使用稀疏注意力提高模型效率的同时，尽可能保证模型的效果。除此之外，本章提出了分类信息层次融合的方法，用以优化分组稀疏注意力模型获取输入序列整体信息的流程，更好地落地于分类任务场景。

第 4 章 基于贡献度的 Token 层间选择策略

4.1 研究动机与方法概述

在第三章中，提出了基于层内分组的稀疏注意力策略。该策略将输入序列按照相关度分为多个分组片段，在各组内部执行互相独立的注意力计算，以此将 Token 的注意力视野聚焦在组内更相关的局部信息上。在此基础上，本章在纵向组间提出全局信息引入策略，用以融合局部和全局的信息，增大注意力视野范围。具体的，对于分组后的序列排列，从每组中选取组内贡献度较高的 Token 作为代表，各组代表拼接到一起作为全局信息。将全局代表的信息融入到组内的局部信息中，两个维度的信息进行综合以缓解分组注意力导致的信息交互损失问题。

模型输入序列的长度通常是一个较大值，在一些简单任务上该值被设置为 128，在以 BERT 为代表的相关模型中该序列长度值会被填充到 512，许多现实场景数据的长度往往比 512 还要长。考虑到在对序列进行分类时直接参考的只是少数关键的全局向量，并非输入序列所有位置的 Token 都对最终类别预测有关键性影响，消耗大量计算资源的过程可能只对预测有微弱作用。本章对输入序列中 Token 的重要性进行量化，在注意力模块和前馈神经网络之间加入序列长度缩减模块。该模块以贡献度量化指标作为参考，选择重要的 Token 进行保留，不重要的信息进行适当丢弃，以此减少计算资源消耗。在模型的纵向层间，随着不同编码器层之间的数据传递，序列的长度也在逐步缩减。模型越深层的位置所需的计算资源和内存占用也越少，整体的模型效率得到了提高。

本章聚焦 Transformer 模型的不同编码器层之间以及不同分组之间，提出多种基于注意力分数的 Token 贡献度计算策略。在此基础上，在分组之间实现局部和全局信息融合；在编码器层之间实现序列长度缩减。序列长度缩减部分的结构如图 4.1 所示，绿色方块对应输入序列 Token，粉色方块对应全局向量，紫色方块对应聚类中心向量，方块的颜色越深表明其贡献度量化值越大。

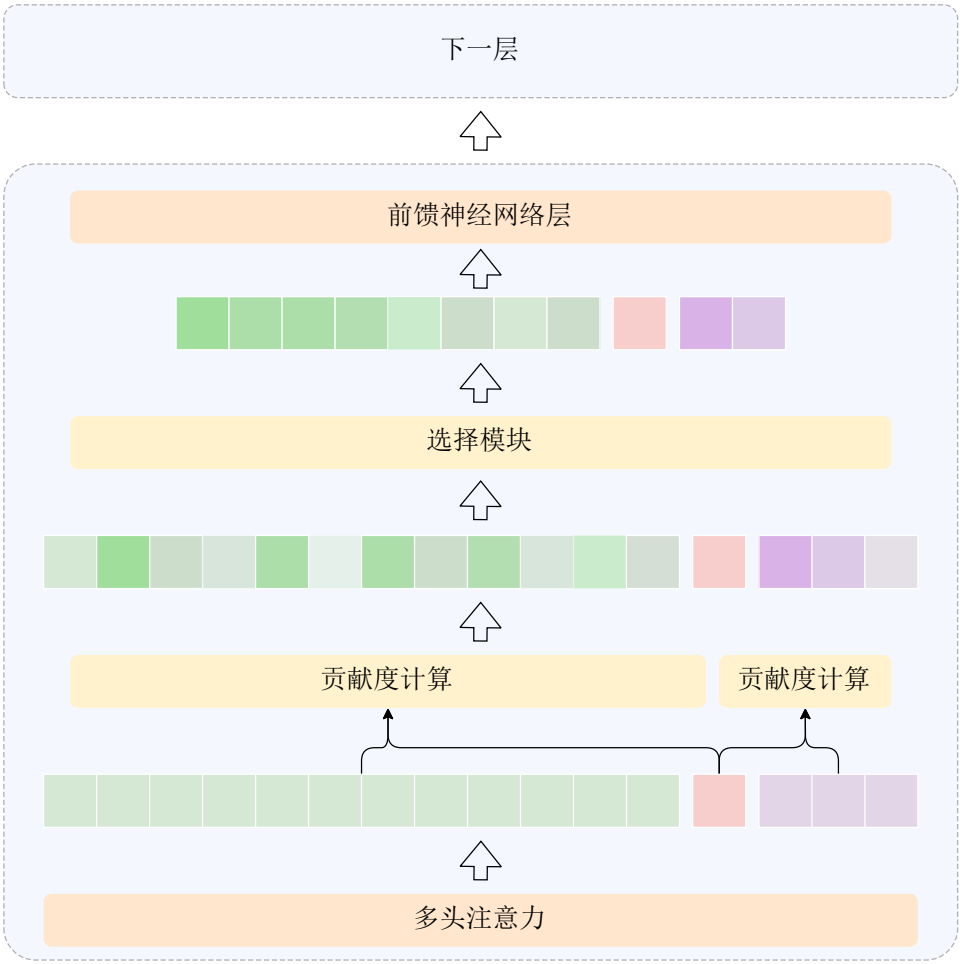


图 4.1 序列长度缩减结构示意图

4.2 Token 贡献度评估策略

在注意力机制的计算过程中，每两个 Token 之间都会通过向量内积计算得到一个注意力分数，所有 Token 之间的注意力分数构成了一个注意力矩阵。注意力分数衡量了两个 Token 的相关性，也影响着不同 Token 间信息传递的程度。基于注意力矩阵，本章针对不同场景设计多种计算策略来评估 Token 的贡献度，进而量化 Token 的重要程度。

4.2.1 Token 的组内贡献度

当视角聚焦于一个分组片段的内部，每个分组片段都包含长度为L_c的子序列，序列的 Token 之间在组内注意力阶段可以计算得到注意力矩阵$ScoreXX_c \in R^{L_c \times L_c}$，$c$ 为所属分组的索引。该矩阵反映的是两个 Token 之间的相关度，对该矩阵在行的维度进行求和，得到$X_{c,i}$和同组其他位置$X_{c,0:L_c}$的注意力分数总和，将

其称为组内贡献度矩阵。其中，i 对应 Token 在组内的索引。

$$SignificanceToken_{c,i} = \sum_{j}^{L_c} ScoreXX_{c,i,j} \qquad (4-1)$$
$$1 \le c \le N_c \quad 1 \le i,j \le L_c$$

某种意义上，$SignificanceToken_{c,i}$越大，说明$X_{c,i}$和同组其他位置 Token 的注意力分数之和越大，通过注意力机制传递的信息也越多，进而可以认为该 Token 在组内影响力更大，或者说更具有代表性。基于贡献度矩阵，可以在每组内部对 Token 进行排序，选择出各组最具有信息影响力的 Token。将各组的代表 Token 提取出来拼接到一起，就构成了全局信息代表。

4.2.2 Token 对中心向量的贡献度

在对输入序列进行分组时，本文使用了基于注意力相似度的分组策略：每个分组对应一个聚类中心向量，输入序列的每个 Token 与各聚类中心向量通过内积计算得到注意力分数，基于该分数决定 Token 所归属的分组。在分组阶段的计算过程中，输入序列的信息会通过注意力机制传递到中心向量中。在分类信息层次融合的流程中，聚类中心向量的信息又会传递到全局分类向量中进行融合。每个分组的中心向量在此结构中承载着信息传递的关键作用，因此基于 Token 与聚类中心向量的注意力分数可以提出一种重要性评估策略。

在模型每一层编码器的序列分组过程中，输入序列会和聚类中心向量进行一次注意力计算，得到各 Token 与各组中心向量的注意力分数矩阵$ScoreCX \in R^{N_c \times L}$。将该注意力矩阵在聚类中心向量的维度进行求和，可以得到每个 Token 和所有中心向量的注意力分数之和，将该分数视作 Token 对中心向量的影响度量化指标，并作为一种贡献度评估策略。

$$SignificanceCenter_i = \sum_{c}^{N_c} ScoreCX_{c,i} \quad 1 \le i \le L \qquad (4-2)$$

其中，$SignificanceCenter_i$表示输入序列中X_i对聚类中心向量的贡献度。基于该贡献度策略选择的 Token 在分类任务中向中心向量传递的信息更多，对最终分类结果的影响更大。

4.2.3 Token 对分类向量的贡献度

与“Token 对中心向量的贡献度”的评估策略类似，本策略也将根据 Token

和其他特殊向量的注意力分数来量化其贡献度。本文在 3.3 节提出了分类信息层次融合的方法来优化分组注意力模型在分类任务中的应用流程，该方法设置了一个全局分类向量，该向量逐层级地获取输入序列中的抽象分类信息，最终直接参与整个序列层级上的分类预测任务。

计算输入序列和全局分类向量的内积，并将该值视作各 Token 对全局分类信息贡献度的量化指标。基于该量化指标在整个输入序列的维度对 Token 进行重要性排序，以此选择出包含更重要信息的 Token。

$$SignificanceClass = \frac{Class^{I} \times X^{C}}{\sqrt{d}} \qquad (4-3)$$

该策略衡量的是输入序列对全局分类信息的影响程度，相比于中心向量贡献度，分类向量贡献度更为直接地反映了 Token 对最终分类结果的影响程度。在本文后续的序列长度缩减部分将会采用该策略。

4.3 局部和全局注意力的信息融合

注意力模块的核心原理是通过注意力计算，使得两个信息源之间能够进行信息的传递交互。以自注意力机制为例，输入序列X通过不同的映射矩阵被转换到多头隐层子空间，形成 Query、Key 和 Value 三个信息表示。Query 从某种程度上可以理解为信息受体，而 Key 和 Value 成对出现，它们的信息来源相同，可以视作信息供体。Query 和 Key 分别作为两个信息源的代表进行注意力内积计算，由此得到注意力矩阵。对该矩阵在行的维度进行 Softmax 函数计算，得到归一化处理后的矩阵，这个矩阵决定了 Query 可以从不同的 Key 所对应的 Value 中获取信息的多少。由此分析，注意力机制可以理解为 Query 从 Key 中获取信息的过程，Query 所能关注到 Key 的数量和范围决定了其所能从其他地方获取信息的数量，也间接影响着模型整体信息流动的程度，进而影响着模型最终的效果。

在层内对输入序列进行聚类分组处理之后，长度为L的输入序列被分为N_c个长度统一为L_c的分组片段。每个分组内部独立地进行注意力机制的计算，组内的 Query 也就只能和组内视野范围的 Key 进行注意力信息交互，也就是只能从同组获取局部信息。然而，标准注意力机制计算的一个特点就是不受距离限制，每个位置的 Token 都可以从其他所有位置获取信息，进行完整充分的信息交互。在层内分组的基础上，本章基于 Token 贡献度评估方法提出全局信息引入策略，在局部分组注意力和标准完整注意力之间进行了模型效率和模型效果的权衡折中。

在具体流程上，首先需要依据贡献度评估策略将各组内部重要的 Token 选择出来。然后，被选择出来的 Token 作为各组代表参与到其他分组的组内注意力计

算过程中，作为全局信息与组内局部信息互为补充。从这个出发点考虑，被选择的 Token 应该拥有更高的组内信息影响力，或者说应该尽可能包含更多的组内信息。因此，该阶段采用 Token 的组内贡献度评估策略作为依据对各组的代表进行选取。具体的，在组内注意力计算过程中，长度为L_c的序列通过多头点积注意力计算会得到局部的注意力分数矩阵$S_{local} \in R^{L_c \times L_c}$。假设每组只选取一个代表参与全局信息补充过程，则得到$K_M, V_M \in R^{N_c \times d}, R^{N_c \times d}$。全局代表和各分组序列进行注意力计算可得到全局的注意力分数矩阵$S_{global} \in R^{L_c \times N_c}$。将局部和全局的注意力分数矩阵在第二个维度拼接到一起，构成综合注意力分数$S_c \in R^{L_c \times (L_c + N_c)}$。随后按照标准计算流程对注意力分数进行 Softmax 归一化处理，将归一化后的分数和拼接得到的 Value 相乘，得到该局部-全局注意力计算过程获取的上下文信息表示。该过程的结构如图 4.2 所示。

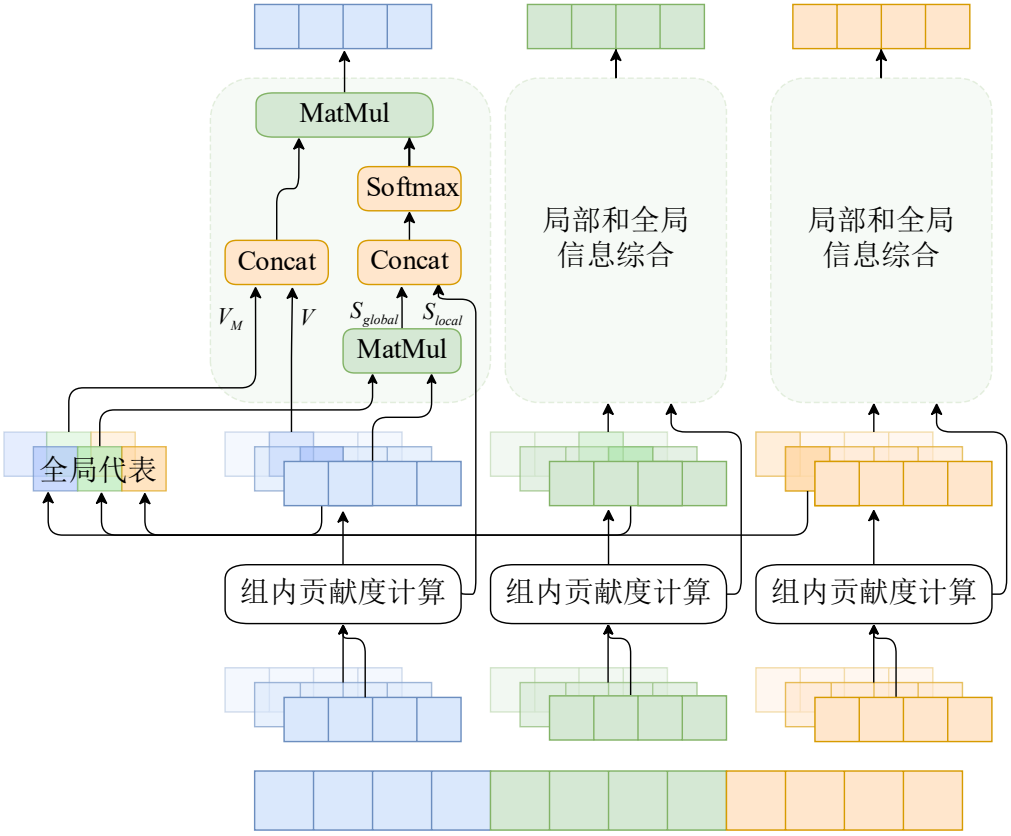


图 4.2 基于选择策略的局部和全局信息融合结构

$$S_{local} = \frac{QK^T}{\sqrt{d}} \tag{4-4}$$

$$S_{global} = \frac{QK_M^T}{\sqrt{d}} \tag{4-5}$$

$$S_c = Concat(S_{local}, S_{global}) \quad (4-6)$$

$$V_c = Concat(V, V_M) \quad (4-7)$$

$$S_c = Softmax(S_c) \quad (4-8)$$

$$Context = S_c V_c \quad (4-9)$$

其中，Q、K、V分别代表组内注意力计算时的输入；K_M和V_M是基于各组组内注意力分数S_{local}选出来的全局代表；$Context$是该注意力计算过程得到的输出结果，它综合了局部和全局的信息。

4.4 层间的序列长度缩减

基于 Transformer 架构的模型，其计算需求主要集中在多头自注意力和前馈神经网络两个模块。假设模型输入序列的长度为L，隐层表示的向量维度为d，前馈神经网络的中间层表示维度为$4d$，则注意力部分的计算复杂度为$O(L^2 d)$，前馈神经网络层的计算复杂度为$O(Ld^2)$。当输入序列的长度逐渐增加时，所需的计算消耗将以平方级的速度随之上升，模型的训练和推理都将耗费更长的时间，这阻碍了模型在部分长序列场景的进一步深度应用。

在分类任务的场景下，3.3 节提出了分类信息层次融合的方法，使得模型可以仅依据最后输出的全局分类向量来进行类别预测。然而在实际的应用场景中，输入序列的长度往往是远大于 1 的，同时并非输入序列的所有位置都会对分类变量产生关键性的影响。基于此提出序列长度缩减策略，该策略主要分为贡献度计算和选择缩减两个阶段。

在贡献度计算阶段，依据 4.2 节的贡献度评估策略对输入序列的重要性进行量化。在具体计算上，首先将全局分类向量和经过分组注意力计算后的序列向量进行注意力计算，以此来得到输入序列与全局分类向量的注意力分数矩阵$SignificanceClass \in R^{1\times L}$。然后，将该注意力矩阵在输入序列的维度按降序进行排序，得到排序后的序列索引，排序后越靠前的位置对分类向量的信息贡献度越大。

在选择缩减阶段，基于上一阶段排序后的索引，选择序列中更为靠前的片段送入下一层编码器中进行处理，其余部分进行丢弃缩减。考虑到第三章在横向层内对序列进行了分组处理，在缩减序列时也需要尽可能保证缩减后的长度能够被预先确定的分组大小所整除。因此，将每层编码器中缩减的长度设置为分组片段长度L_c的整数倍，同时将每层缩减的组的数量设置为超参数N_{short}。通过模型输入序列的长度L和分组中心大小N_c来动态计算序列缩减数量N_{short}。最终使得每

层模型缩减的数量是一致的且尽可能多地对整个输入序列的长度进行缩减，具体应该满足如下公式。

$$L = L_{left} + N_{layer} \times N_{short} \times L_c \quad L_{left} \geq 0 \qquad (4-10)$$

其中，N_{layer}为编码器或解码器的层数；L_{left}是经过缩减处理后剩余序列的长度。在启用了序列长度缩减模块时，对于排序后的序列索引，将其尾部长度为$N_{short} \times L_c$的片段进行删除。在此之后，模型通过注意力机制获得的上下文信息X^h经过 Dropout 处理，再与原始输入序列X通过残差连接的形式进行综合，得到该层编码器的输出X^O。对于长度为L的原始输出X^O，使用上一步经过缩减处理后的序列索引来从原始输出中获取对应位置的向量，并拼接到一起得到长度缩减后的输出X^{Short}。除了对输入序列进行长度缩减处理，也需要根据聚类中心对全局分类向量的信息贡献度对其同步进行长度缩减。当序列长度缩减$N_{short} \times N_c$时，聚类中心也需要同步缩减N_{short}的长度，方能保证在整个模型的运行过程中，可以根据聚类中心向量将输入序列平均分成预设的统一长度。如果输入的掩蔽矩阵存在，也需要参考输入序列排序后的序列索引对掩蔽矩阵进行同样的缩减处理。最后，将经过缩减处理后的序列向量表示、聚类中心向量表示和掩蔽矩阵作为结果送入到前馈神经网络层进行处理，得到本层编码器或解码器的输出。

4.5 复杂度分析

在使用了序列长度缩减策略之后，Transformer 模型中每层编码器或解码器内部的输入序列长度都不一样，随着模型层数由浅及深，输入长度逐渐缩短，消耗的计算也更少。因而，单独计算一层编码器的复杂度难以在理论层面衡量整个模型的计算消耗和内存占用。为了便于比较，在此假设模型层数统一为N_{layer}。根据 3.4 节的分析，单独一层编码器的复杂度为$O(LN_c d + LL_c d + N_c d)$。假设原始输入序列长度能够被缩减比例所整除，且到最后一层模型时所有信息都已融合到全局分类向量中，此时序列长度为 0。由此可以得到所有编码器层的计算消耗总和为$L(N_{layer}+1)/2 * (N_c d + L_c d) + N_c d N_{layer}$，平均到每一层计算消耗为$L(N_{layer}+1)/(2N_{layer}) * (N_c d + L_c d) + N_c d$。

如果将每个聚类分组的大小设置为$L_c = \sqrt{L}$，则可以将第 3 章部分模型的复杂度简化为$O((2L+1)\sqrt{L}d)$，在应用第 4 章序列长度缩减策略后的模型复杂度可以简化为$O((L+1)\sqrt{L}d)$。通过对比可以发现，虽然两者在计算量级上一致，但后者模型消耗的计算总量得到了大幅下降。

4.6 本章小结

本章主要从纵向层间的视角来分析 Transformer 模型，针对多层堆叠网络的层与层之间以及不同的分组之间设计了相关优化策略。首先，本章基于注意力分数矩阵设计了 Token 的组内贡献度、Token 对中心向量的贡献度和 Token 对分类向量的贡献度三种评估策略，以此对 Token 的重要性进行量化。基于贡献度评估方法，本章提出了两种应用策略。其一是全局信息的引入：选择各组组内贡献度最高的 Token 作为代表，将所有代表拼接到一起作为全局信息，将该信息引入到各分组的组内注意力计算中，以此弥补分组注意力带来的信息交互损失。其二是层间序列长度缩减：通过贡献度评估策略选择输入序列中对分类信息贡献更大的 Token，对此部分信息进行保留，不重要的进行适当丢弃，以此实现对序列长度的缩减，减少模型计算消耗。两个策略中，前者重在对第三章提出的分组注意力进行补充，以提高模型效果。后者核心在于牺牲部分不重要的信息，以提高模型效率。多种策略综合，在保证模型效果的同时尽可能提高模型效率，实现两者的权衡折中。

第 5 章 实验设计与分析

为了验证本文方法的有效性和高效性，本章开展一系列相关实验。首先，本章从不同优化策略的典型代表中选择几个基准模型，并针对自然语言处理领域的文本分类任务和词级别分类任务进行了实验。随后，针对初始聚类中心生成等策略，进行消融实验来验证优化策略的有效性。接着，开展参数实验来探究关键参数的设置对模型效果和效率的影响。最后，通过可视化实验对先前的猜想和假设进行验证，同时探究序列长度缩减模块的作用原理。为了公平比较，本文模型和基线模型的超参数设置都尽可能保持一致。模型的结构方面除了注意力模块之外的其他模块也都保持一致。

5.1 实验设置

5.1.1 基线模型

本文选取若干个模型作为基线参考。这些模型或是与本文处理策略较为相似，或是不同优化策略的代表性模型，这些模型如下所示。

Transformer[1]：使用完整注意力计算的标准模型结构。

Reformer[17](ICLR2020)：该模型使用局部敏感哈希将输入向量映射到多个桶，在每个桶内进行注意力计算，将模型的计算复杂度从$O(L^2)$降低到$O(L\ log\ L)$，其中L为序列长度。此外，该模型使用可逆残差层代替标准残差，这允许在训练过程中只存储一次激活值而不是 N 次，其中 N 是模型的层数。如此在不增加计算复杂度的情况下可以实现更深的神经网络。

Routing Transformer[18](TACL2021)：该模型结合了基于内容的稀疏注意力方法的建模灵活性和基于局部时间稀疏注意力方法的效率收益。使用了基于在线 K-Means 的稀疏模块，动态地调整每个词嵌入向量与其相邻向量的交互强度，将注意力的整体复杂度从$O(L^2d)$降低到$O(L^{1.5}d)$。

ClusterFormer[19](ACL2022)：该模型与本文使用的分组策略最为相近，同样使用了神经聚类方法对输入序列进行分组，聚类任务和目标任务可以共同训练。为了解决各分组 Token 数量不一致的问题，该模型对 Key-Value 对进行了填充处理，使得 Query 可以和相邻两个组的 Key-Value 对进行注意力信息交互。除此之外，它还引入了两个额外的损失计算来源，用以提高分类效果。

5.1.2 实验任务和数据集

文本分类任务。文本分类是针对一个给定的文本，为其分配一个标签或类别的任务。一些常见的文本分类任务有情感分析、自然语言推理和评估语法正确性等。该任务能够反应模型对整个输入序列进行信息捕获的能力。对于该任务，本文分别选取了基准模型论文中提到的数据集和当前相关领域中较为常用的数据集用于测试。这些数据集如下所示。

20NEWS[40]：一个用于文本分类、文本挖掘和信息检索研究的国际标准数据集。该数据集收集了大约 20000 个新闻组文档，均匀分为 20 个不同主题的新闻组集合。数据集类别有 20 类，总样本数为 18846。

SUBJ：一个由电影评论文件构成的集合，根据其总体情感极性（积极或消极）或者主观评价进行标注，并根据其主观状态（主观或客观）或极性进行标注的句子。

MR[41]：一个电影评论数据集，分为正面和负面两类。

IMDB[42]：该数据集包含来自互联网电影数据库的 50000 条两级分化的评论。数据集被分为用于训练的 25000 条评论与用于测试的 25000 条评论，二者都包含 50%的正面评论和 50%的负面评论。

emotion[43]：该数据集取自 tweet_eval 的子任务，后者由 Twitter 数据的七个异质任务组成，每一个任务都是多类别推文分类结构。这些任务包括讽刺、仇恨、攻击性、立场、表情符号、情绪和情感，每个数据集都以相同的格式呈现，并且有固定的训练集、验证集和测试集。如不特殊说明，后文将以 emotion 指代 tweet_eval 的情感子任务。

MIND[44]：一个由微软推出的咨询推荐类的大规模数据集，内容来源于 Microsoft 新闻网站的匿名日志。数据包含了新闻 ID、类别、标题、摘要等信息，本文选择数据集中的标题和类别信息，用于文本分类任务的实验。

上述数据集的部分统计结果如表 5.1 所示。

词级别分类任务。该任务是自然语言处理中的常见任务，目的是对一段文本中的部分 Token 给出类别标注。一些代表性的 Token 级别分类任务有命名实体识别（NER）和词性标注（PoS）等。命名实体识别用于识别文本中的特定实体，比如日期、人名和地名等。词性标注会判断一段文本中哪些词是名词，哪些是动词等。该类型任务可以检验模型对输入序列中单个 Token 的分类能力，也是对模型内部信息交互能力的验证。针对该任务，本文选取了 CoNLL-2003 数据集[45]，该数据集内部包含三个子任务：命名实体识别、词性标注和组块标注。

表 5.1 数据集相关参数统计

数据集	训练集	验证集	测试集	平均长度	类别数
20NEWS	11314	7532	7532	326.5	20
SUBJ	9000	1000	1000	23.35	2
MR	9594	1068	1068	20.39	2
IMDB	25000	5000	25000	279	2
emotion	3257	374	1421	20	4
MIND	101527	20000	120959	11	18

数据集的分割设置。如果数据集本身就提供了训练集、验证集和测试集的分割，则使用数据集的原始分割。如果数据集只提供了训练集和测试集，则从测试集中随机选取部分样本作为验证集。通过上述处理，本文在所有数据集上的实验都能保持相同的训练方案。

5.1.3 参数设置

在本文的实验中，使用维度为 300 的 Glove 预训练词向量嵌入来初始化不同位置 Token 的向量表示。随机数生成种子设置为 666，这能确保部分随机初始化的参数在每次实验过程都保持一致。

在数据集 20NEWS、SUBJ 和 MR 的实验中，本文的参数设置与基准模型的设置保持一致：模型的批处理大小为 128。预训练词向量维度和模型隐层特征维度都设为 300。前馈神经网络中间层的维度设为 1024。模型反向传播过程的优化器使用 Adam，相对应的学习率设置为 5e-5。模型 Dropout 层的概率设为 0.2。整体结构上，模型的层数设置为 2，多头子空间的数量设置为 4。聚类分组的大小按照不同数据集的最大长度，统一设置为最大长度的平方根进行取整所得到的整数值。注意力分组误差对应的正则化参数设为 1。在序列缩减参数的设置上，本文在满足公式约束的条件下尽可能减小 c 的取值，以达到尽可能大的缩减比例。

在本文新增的补充数据集上，包括 IMDB、MIND、 emotion 和 CoNLL-2003 的实验过程中，对部分参数进行了调整。模型的批处理大小设为 32。输入序列的隐层特征维度设为 512。前馈神经网络层的中间维度参考通用经验，设为隐层特征维度的四倍：2048。模型中编码器的层数设置为 4。对于模型输入数据的最大长度统一设置为 512，相对应的分组片段的视野范围统一设置为 32。除此之外的其他参数与上一部分的设置保持一致。

在针对不同数据集的训练过程中，本文都使用了提早停止策略对训练程度进

行控制。对于每个数据集在不同模型上的实验，本文按照相同的策略和比例从测试集中选取部分或全部作为验证集。在模型训练过程的每个轮次完成后，在验证集上运行模型得到准确率等指标，将当前指标与记录的最佳指标进行比较，如果在超过指定数量的轮次后最佳指标没有提升，则提前终止模型训练过程。在模型训练饱和之后，加载最佳指标所对应的模型检查点，在测试集上进行模型测试，得到最终的模型效果衡量指标。针对不同的数据集，最大训练轮次和提早结束轮次设置也不同，具体设置根据在数据集上的初步简单实验情况来确定。本文所采用的训练轮次方案主要有两种：最大训练轮次 60，提早结束轮次为 10；最大训练轮次 10，提早结束轮次为 3。

5.1.4 评估指标

对于模型效果的衡量，本文参考了相关数据集的通用指标，以准确率来作为效果指标。对于模型效率的衡量，本文主要从运行时间和内存占用两个维度入手，分析不同模型在相同的硬件设备下的运行情况。

高效率模型变种的研究一方面是为了优化模型结构，用更少的计算来尽可能完整地完成模型参数学习的过程。另一方面也是为了能减少模型训练过程的时间花费和计算资源消耗，提高推理过程的计算速度，降低模型应用到实际场景的门槛。因此，本文还使用了综合指标来衡量变种模型在效果和效率上的提升。具体实现上，本文在每个训练轮次完成后，在验证集上运行模型，同时记录该训练轮次所耗费的时间和阶段模型在验证集上的效果指标。将每次耗费的时间累加到训练总时间中，记录训练总时间的阶段性节点值。将效果指标与当前最优指标进行比较，更新最优指标并记录最优指标节点值。在整个训练过程结束后，将训练时间和阶段最优指标的记录分别作为横轴和纵轴绘制趋势变化曲线。该曲线反映了模型在验证集上的效果随着训练时间的变化趋势。作为一个综合性的评估指标，它一定程度上反映了模型的学习能力。

5.1.5 实验环境

在模型的编码实现上，本文使用 PyTorch 框架[46]作为训练推理平台。基线模型的代码引用自 ClusterFormer 论文提供的源代码 [2]。除了将单词级别分类任务和句子级别分类任务的输出层封装到一起的改造，本文未对基线模型做任何修改。

在训练条件上，本文所使用的相关硬件配置如表 5.2 所示：

[2] https://aclanthology.org/attachments/2022.acl-long.170.software.zip

表 5.2 实验硬件环境

操作系统	Ubuntu Server 18.04.5 LTS
CPU	Intel(R) Xeon(R) CPU E5-2620 v4 @ 2.10GHz
GPU	TITAN V (12G)
软件版本	CUDA 10.2 Python 3.6.4 Torch 1.10.2

本文的相关实验只使用了一块 GPU 加速卡，在比较模型效率时确保了只有当前模型训练程序在该 GPU 上运行，不存在其他运行任务对本文实验任务产生干扰的情况。

5.2 语言分类任务中的性能与分析

5.2.1 文本分类任务

本文在若干个文本分类数据集上验证了模型的效果。在后文的表述中，用 CSformer 指代本文的模型，意为 Cluster 和 Select。由于序列长度缩减比例的设置随着数据集平均长度和输入序列长度的不同会有多种取值，因此在该部分实验中选取两个端点值作为代表：CSformer 对应序列长度缩减比例为 0 时的模型；CSformerShort 对应序列长度缩减比例设置为最大时的模型。各个模型在数据集上的效果如表 5.3 所示。

表 5.3 模型在文本分类数据集上的准确率统计

模型	20NEWS	SUBJ	MR	IMDB	emotion	MIND
Transformer	82.2	94.6	79.8	84.27	74.24	**76.45**
Reformer	80.9	95.1	79.9	80.40	56.23	58.86
Routing Transformer	80.9	95.0	80.1	63.77	39.48	56.03
ClusterFormer	79.0	94.5	79.3	74.47	39.27	72.67
CSformer	**82.6**	**95.6**	**81.2**	85.19	**75.58**	75.21
CSformerShort	81.9	95.1	81.1	**86.40**	74.31	73.99

通过对实验结果进行观察分析可以发现如下现象：

（1）在数据集 20NEWS、SUBJ、MR、IMDB 和 emotion 上的结果表明，本文的模型在效果上普遍优于基线模型。与其他模型相比，本文的模型在 20NEWS 等 5 个数据集上分别有至少 0.4%、0.5%、1.1%、0.92%、1.34%的效果提升。

（2）在数据集 IMDB、emotion 和 MIND 上，本文模型相比于 Routing Transformer、ClusterFormer 和 Reformer 在部分效果上有明显提升。猜测可能是由于引入了分类信息层次融合的方法，使得文本的分类信息更集中于特定的分类向量中，避免了将所有 Token 的平均作为分类信息时潜在的信息干扰问题。

（3）数据集 MIND 上的结果表明，虽然使用了若干策略对分组稀疏注意力进行优化，但在个别场景下还是避免不了产生少许效果损失。

（4）将序列长度缩减策略作为单独的模块叠加到现有模型上，并将缩减比例设置为尽可能大。从模型效果上来看，序列长度缩减后丢失了部分信息，在多数数据集上相比于不进行缩减的原模型都有了一定程度的下降。然而，在整体效果上相比于其他基线模型还是普遍有所提升。

（5）在数据集 IMDB 上，CSformerShort 的效果比 CSformer 要更优，综合分析可能有两方面的原因：缩减比例设置为最大时，模型舍弃了许多冗余信息，泛化能力得到了提高；IMDB 为二分类任务，其预测更容易受到输入序列中无关信息的干扰。

在模型效率层面，本文记录了模型训练过程中每个训练轮次的平均训练耗时以及 GPU 内存占用的大小，结果如表 5.4 所示。将时间进行归一化处理作为横轴，内存占用作为纵轴，模型在各数据集上的结果如图 5.1 所示。

表 5.4 模型在文本分类数据集上的内存占用与运行时间统计

模型	20NEWS	SUBJ	MR	IMDB	emotion	MIND
Reformer	11213M	6799M	5737M	9923M	9921M	9923M
	27.4s	7.46s	4.96s	211.38s	28.14s	85.61s
Routing Transformer	7613M	5623M	5107M	6549M	6549M	6549M
	17.44s	5.02s	3.64s	160.40s	21.12s	63.75s
ClusterFormer	8627M	5907M	5259M	7479M	7479M	7479M
	17.56s	5.15s	3.71s	153.83s	20.61s	61.28s
CSformer	7523M	5789M	5225M	7097M	7097M	7097M
	12.11s	4.20s	**3.27s**	129.64s	17.42s	52.78s
CSformerShort	**6855M**	**5001M**	**4617M**	**6001M**	**6001M**	**6001M**
	11.57s	**3.66s**	3.56s	**116.36s**	**11.99s**	**38.61s**

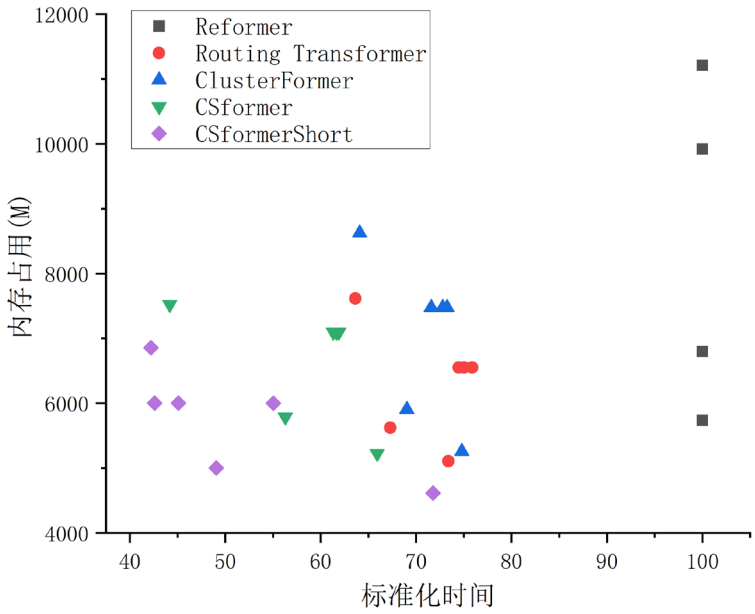


图 5.1 模型在文本分类任务上的效率对比

从表和图所示结果可以得到如下结论：

（1）在模型训练的时间消耗上，本文基础模型相比于其他基线模型所需的时间更少。当使用缩减策略后，模型运行时间有了进一步下降。

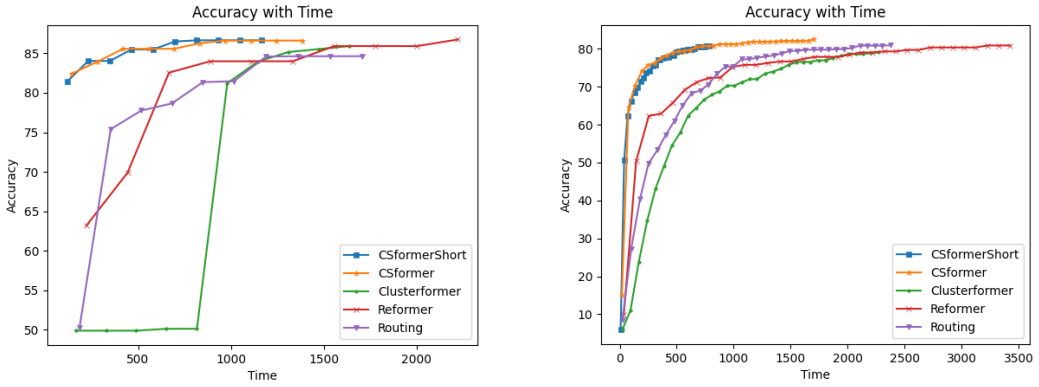
（2）在模型训练的内存占用上，本文基础模型占用的显存比 Reformer 和 ClusterFormer 要更少，比 Routing Transformer 显存占用要多一些。在使用缩减策略之后，本文模型在所有数据集上的内存占用都达到了最少。

（3）在数据集 IMDB、emotion 和 MIND 的实验中，由于模型输入数据的长度被统一设置为 512，因此模型占用内存的大小几乎相等。在运行时间上，由于数据集大小的不同，每个训练轮次所需的时间有所差异。

（4）本文模型在图中的数据点集中于左下方，在内存占用和运行时间上都具有一定优势。

（5）从实验结果可以得知，相同的理论复杂度量级并不等同于相同的执行时间，模型实际执行时间要考虑复杂度中的参数因数和常数项。

除了基础的实验指标之外，本文还记录了模型在验证集上的准确率随训练时间的变化曲线。由于部分数据集训练后期接近于饱和，数据点过于密集难以清晰查看变化趋势。故本文选取了 IMDB 和 20NEWS 数据集作为代表，并根据实际情况调整了训练点的显示密度以更好地反应趋势变化情况。该部分结果如图 5.2 所示。



（a）IMDB 数据集　　（b）20NEWS 数据集

图 5.2 模型准确率随训练时间的变化趋势

从实验结果反映的曲线可以看出，在相同的训练时间内，本文的模型指标增长更快，达到的模型效果更好。若要达成相同的训练效果，本文的模型所需的训练时间更少。一定程度上可以认为本文模型的学习收敛速度更快、效率更高。

综合相关实验结果，本文所提出的模型无论在模型效果层面还是模型效率层面都相较于当前的基线模型有一定提升。应用序列长度缩减策略之后，模型效果上有些许损失，但在模型效率上有了进一步提高。在实际应用时，如果具体场景对模型效果要求更高，则可降低缩减比例或不缩减以保证效果。如果对模型效率要求更高，则可适当增大缩减比例以提高效率。通过对模型参数进行设置在效果和效率之间进行权衡调整。

5.2.2 词级别分类任务

为了验证模型在注意力信息交互上的有效性，本文在词级别的分类任务上选取了数据集 CoNLL-2003，并在其提供的命名实体识别、词性标注和组块标注三个分类任务上进行了相关实验。考虑到这些任务通过少次的训练即可达到理想的模型效果，过多的训练对于效果的提升非常微小。因此在该部分实验中，将各模型统一训练 10 个轮次，然后在最佳保存点上进行模型效果测试，实验结果如表 5.5 和图 5.3 所示。

表 5.5 模型在词级别分类任务上的准确率统计

模型	NER	POS	CHUNK	AVG
Reformer	99.55	98.11	98.38	98.680
Routing Transformer	99.54	98.04	98.43	98.670
ClusterFormer	99.78	99.62	**99.45**	99.617
CSformer	**99.81**	**99.63**	99.44	**99.620**

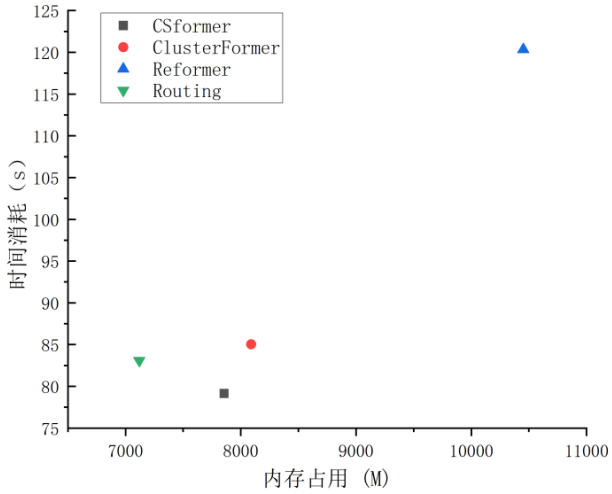


图 5.3 模型在词级别分类任务上的效率对比

通过对实验结果进行观察分析，可以发现本文模型在该任务上的效果整体略优于其他分组稀疏基线模型，在模型效率上也相对于其他基线模型普遍有些许提升。

5.3 消融实验

为了提高层内聚类分组阶段的准确性，本文在前期和后期分别提出了多种策略。在分组前期，提出了初始聚类中心生成策略。在分组后期，使用了聚类中心信息增强策略，同时提出了分组损失计算方法用于训练注意力分组模块。为了弥补分组稀疏注意力导致的信息交互损失问题，本文基于贡献度选择策略提出了全局信息补充策略。本节选取了若干个多分类任务的数据集，针对上述模块进行消融实验来探究对模型效果的影响。

5.3.1 聚类中心生成算法

对于初始聚类中心的生成过程，本文设计了基于距离的平均生成策略和基于K-Means++的采样生成策略。为了验证上述两种策略的有效性，本文以ClusterFormer 论文中的初始化策略作为基准参照，在此基础上控制其他实验变量一致，得到的实验结果如表 5.6 所示。

表 5.6 模型在使用三种不同生成策略下的准确率对比

初始化策略	emotion	MIND	IMDB
ClusterFormer 方案	74.88	73.67	84.74
基于 K-Means++采样生成	75.58	**75.21**	84.47
基于距离平均生成	**75.65**	74.02	**85.19**

由实验结果可以看出，本文的两种初始聚类中心生成策略在不同数据集上对模型效果的提升均有一定贡献。在数据集 emotion、MIND 和 IMDB 上的结果显示，相比于 ClusterFormer 的聚类中心初始化方案，应用本文的两种生成策略后模型效果最高分别提升了 0.77%、1.54%和 0.48%。本文的两种生成策略从输入序列中获取了信息，理论上在第一层对输入序列进行分组时准确性更高，进而提高了模型效果。

5.3.2 全局信息引入策略

针对分组注意力机制导致的信息交互损失问题，本文设计了基于贡献度评估策略的全局信息弥补策略，以此将全局信息引入到组内注意力计算中，局部和全局的信息互相结合来尽可能对完整注意力机制进行近似模拟。为了验证该模块的有效性，本文在若干多分类数据集上进行了实验，比较了是否引入全局信息对模型效果的影响，具体结果如表 5.7 所示。

表 5.7 模型在是/否引入全局信息下的准确率对比

是否引入全局信息	emotion	MIND	20NEWS
否	75.37	74.47	82.3
是	**75.58**	**75.21**	**82.6**

通过表 5.7 所示的实验结果可以发现，全局信息的引入能够提高模型的效果，在三个数据集上的准确率分别提高了 0.21%、0.74%、0.3%。引入全局信息后输

入序列中的 Token 既能从同组组内获取信息，也能从其他各组的代表中获取信息，因而能达到更广的视野范围。与此同时，视野范围的增大不可避免地带来了更多的计算开销，模型的一次运行需要花费更多的时间。在侧重于模型效果的场景中，可以引入全局信息并适当增加引入信息的数量，以保证预期的效果。

5.3.3 分组误差弥补的策略组合

针对通过聚类中心进行分组可能带来的分组误差问题，本文设计了聚类中心信息增强策略和额外的分组损失计算策略，分别用以强化聚类中心的信息获取能力，以及优化基于传递性注意力的分组策略。针对如上两种策略的组合，本文在若干多分类数据集上进行了相关实验，以模型的准确率作为衡量指标，具体结果如表 5.8 所示。

表 5.8 模型在是/否使用信息增强/分组损失下的准确率对比

是/否采用 信息增强/分组损失	emotion	MIND	SUBJ
否/否	73.61	66.91	95.5
是/否	**75.58**	75.09	94.9
否/是	75.09	75.14	94.8
是/是	75.23	**75.21**	**95.6**

通过对实验结果的分析可以得到如下结论：

（1）两种策略均对模型效果的提升有一定作用。同时应用两种策略时，模型在 emotion 等三个数据集上分别有 1.62%、0.47%、0.1%的效果提升。

（2）在数据集 emotion 上的结果显示，同时应用两种策略并不一定取得最佳效果，但两种策略都对模型效果的提升有一定帮助。

（3）综合考虑数据集本身的复杂程度与达到训练饱和的难度，两种策略对模型效果的提升具有一定的促进作用。

5.4 参数实验

5.4.1 视野范围大小

本文在横向层内对输入序列进行了聚类分组的处理，同时提出了优先确定分组片段长度也就是视野范围的策略。视野范围越小，每个分组内部所包含 Token

的数量越少，局部信息就越少，相对应的分组的数量就越多，全局信息就越多，整体的计算消耗较少。视野范围越大，每个分组片段的长度就越长，相对应的计算消耗越多。通过将视野范围大小设置为不同的值，可以调控组内局部的视野范围，同时也间接影响全局信息的数量。

为了探究视野范围大小对实验结果的影响，本文以输入序列最大长度的平方根取整作为主要参考值，在参考值附近以较小步幅选取了若干参数值，同时也选取了若干差值较大的参数值。本文在不同参数值的设置上进行了对照实验，实验结果如表 5.9 所示。

表 5.9 模型在不同视野范围大小下的准确率和时间消耗对比

视野范围	emotion		MIND	
	准确率(%)	时间消耗(s)	准确率(%)	时间消耗(s)
16	74.74	19.15	71.47	566.95
28	75.58	19.14	71.39	567.80
32	75.58	18.72	71.52	**547.28**
36	**76.00**	19.15	70.93	573.89
64	74.88	18.25	**71.56**	547.94
128	74.95	**18.12**	71.22	554.40

通过对实验结果的分析观察，可以得到如下结论：

（1）根据 emotion 数据集在视野范围从 16~64 的效果变化来看，视野范围并不是越大越好，也不是越小越好，该参数在一定范围内存在较为合适的值。MIND 数据集的效果变化有所波动，但视野范围处于中间的取值在效果上也整体略优于两端的取值。

（2）由于引入了全局信息，当组内局部视野范围减小时分组的数量会增多，进而全局代表会随之增多。因此视野范围的改变一定程度上也影响局部和全局信息的比例。这对模型效果和模型效率都有一定程度的影响。

5.4.2 序列长度缩减比例

本文在纵向层间对输入序列按照贡献度进行了排序筛选，在每层的注意力模块之后会按照分组长度的整数倍N_{short}消除不重要的 Token。消除的比例越大，损失的信息就越多，随着序列长度的缩短，模型所需的计算资源消耗也随之减少。消除的比例越小，保留下来的信息越多，相对应需要的计算量也越大。为了探究缩减比例对模型效果和模型效率的影响，本文在不同缩减比例参数下进行了相关

实验。实验结果如表 5.10 所示。

表 5.10 模型在不同缩减比例下的准确率和时间消耗

缩减比例 N_{short}	emotion		IMDB		20NEWS	
	准确率(%)	时间(s)	准确率(%)	时间(s)	准确率(%)	时间(s)
1	74.24	18.35	84.48	127.59	82.4	13.83
2	**74.31**	16.46	85.25	115.18	**82.6**	13.23
3	**74.31**	14.72	85.52	104.11	82.0	12.75
4	**74.31**	**12.08**	**85.66**	**91.23**	82.0	12.46
5	—	—	—	—	82.1	**12.43**

通过分析实验结果，可以得到如下结论：

（1）在 20NEWS 数据集上，随着缩减比例的增加，准确率呈现先上升后下降的趋势。该数据集平均长度较长，在缩减比例较小时，随着对少许冗余信息进行删除，模型的泛化能力得到了提高。然而随着缩减比例增加到一定程度，越来越多的信息被丢弃，这其中也包括了部分有用信息，信息的丢失影响了模型效果。

（2）在 emotion 和 IMDB 数据集上，随着缩减比例的增加，模型效果较为平稳，甚至有一定的波动提升。综合分析，上述两个数据集平均长度较短，输入序列被填充到统一长度时存在过多占位字符。其中包含的冗余信息可能会对整体分类产生干扰，对部分填充 Token 进行删除可以使得模型具有更好的泛化能力。特别的，在二分类任务数据集 IMDB 上效果提升程度更为明显，这也印证了冗余信息的干扰问题。

（3）随着序列缩减比例的增加，模型所需计算消耗更少，在运行时间上统一表现出了下降的趋势，这说明缩减策略提高了模型效率。

5.4.3 输入序列的长度

基于标准 Transformer 架构的模型由于其输入序列长度平方级的计算复杂度，在一些长序列数据上需要耗费大量的计算资源，这限制了模型在相关场景的深入应用。本文的模型在层内应用了分组注意力机制，在层间进行了序列长度缩减处理，理论上在长序列数据上会具有更高的模型效率。因此，本文在长序列数据集上进行了参数实验，通过改变模型输入序列的最大长度，来比较不同模型在不同场景下的效果和效率。该部分实验选择了 20NEWS 数据集，它拥有更长的平均文本长度（326）和更多的分类类别数（20）。该实验将标准 Transformer 和本文

模型进行了对比，记录相同训练轮次下各自所获得的准确率和每次训练所需的时间，实验结果如表 5.11 所示，趋势变化如图 5.4 所示。

表 5.11 模型在不同输入序列长度下的准确率和时间消耗对比

输入序列长度	CSformer		Transformer	
	准确率(%)	时间消耗(s)	准确率(%)	时间消耗(s)
128	80.6	**6.11**	80.1	**6.58**
256	81.5	9.12	81.4	10.39
512	**82.0**	13.96	**82.0**	19.62

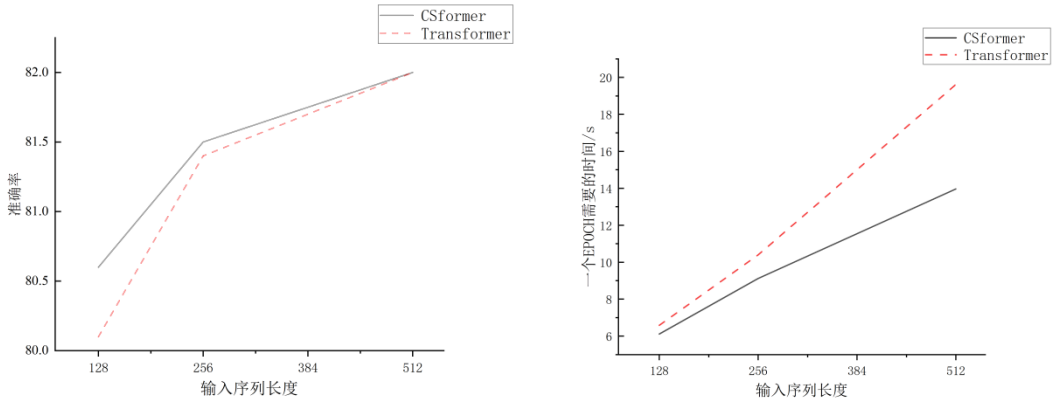


图 5.4 模型效果和效率随输入序列长度的变化趋势

结合表中数据和图中折线所展示的结果，可以得出如下结论：

（1）随着序列长度的增加，模型能获取到更多的信息，模型效果也呈现上升趋势。

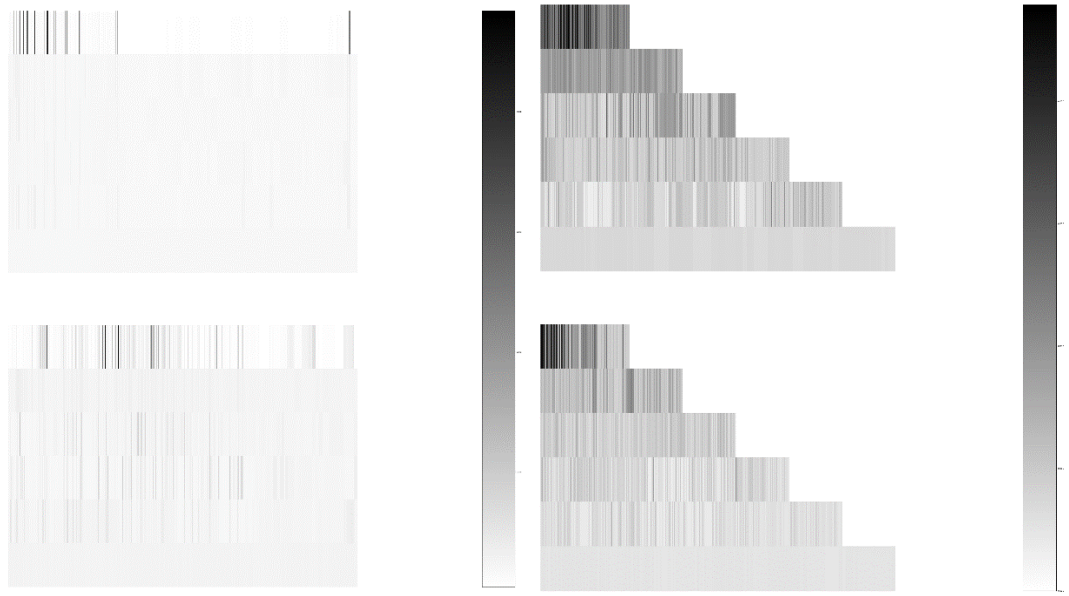
（2）随着序列长度的增加，模型需要更多的计算操作，进而消耗更多的时间。相比于标准 Transformer，本文的模型结构在时间消耗上的增长更慢。这验证了本文模型在长序列数据集上的优势。

（3）相比于标准 Transformer，本文的模型在相同的序列长度下能获得更好的模型效果，计算所消耗的时间也更少，这证实了本文模型在效果和效率上的优势。

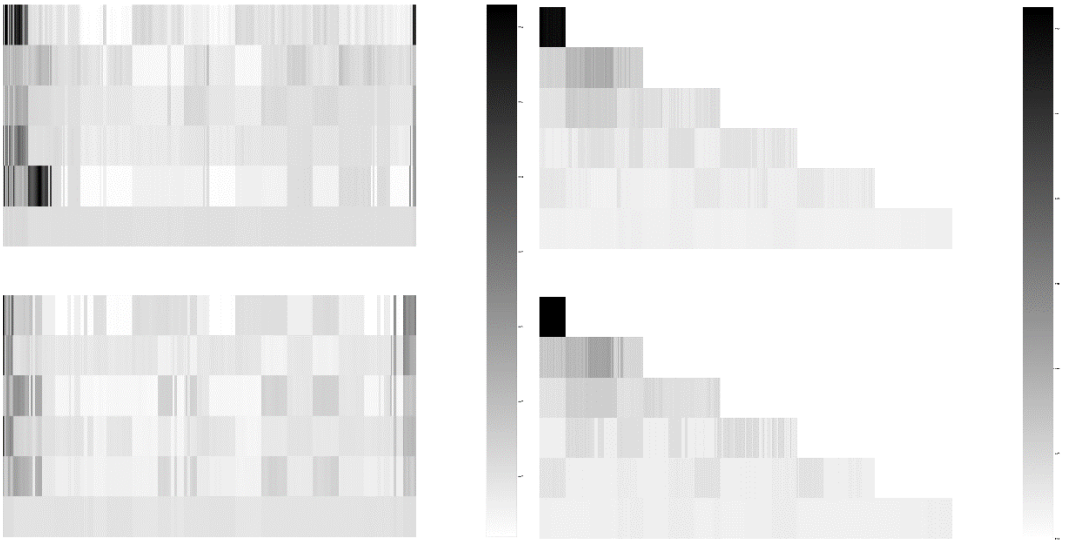
5.5 可视化实验

本文第四章假设并非输入序列中所有 Token 都对分类结果有关键性影响，并基于该假设使用了序列长度缩减策略，将部分贡献度较小的 Token 进行了丢弃。为了探究全局分类向量的作用原理，同时验证上述猜想的合理性，本节进行了可

视化实验。首先分别选取了平均长度短和长的数据集代表：emotion 和 20NEWS，在数据集上对 CSformer 和 CSformerShort 进行了训练。在实验参数的设置上，为了能更清晰地观察层与层之间的参数差异，本实验将模型的编码器层数增加为 6，其他参数与先前的设置保持一致。基于训练好的模型，在测试集上运行模型，在该过程中同时记录模型每层编码器中输入序列和全局分类向量的注意力分数。为了便于结果展示，对每个数据集只选取两个样本，每个样本尽可能从不同批次数据中进行选取。使用工具对注意力分数进行可视化，结果如图 5.5 所示。



（a）CSformer+20NEWS　　（b）CSformerShort+20NEWS



（c）CSformer+emotion　　（d）CSformerShort+emotion

图 5.5 输入序列与分类向量的注意力分数可视化

图 5.5 的子图（a）（b）分别代表 CSformer 和 CSformerShort 模型在 20NEWS 数据集上的可视化结果。子图（c）（d）对应两个模型在数据集 emotion 上的结果。每个子图左侧上下两个热力图对应两个不同的输入样本。在每个热力图中从下往上对应模型层数从第一层到最后一层，横向对应输入序列从左到右。图中颜色越深对应的值越大，颜色越浅对应的值越小。通过对图示结果的分析，可以得到如下结论。

（1）在 CSformer 的可视化实验中，两个数据集都呈现出一些公共现象：整个输入序列中只有部分关键 Token 呈现深色，且随着模型层数递增颜色逐渐变得更深。这在一定程度上说明，并非输入序列所有位置都对分类向量产生了关键影响，同时靠近模型输出层的位置对分类结果的影响相对要更大。

（2）在 CSformerShort 的结果中，随着模型深度递增，深颜色的模块更集中于保留的片段中，且颜色越来越深。这说明模型中的分类信息随着选择与缩减过程的处理逐渐汇集到了部分关键 Token 中。

（3）在不同长度数据集的对比中，短文本数据集的深色部分往往分布在序列靠前的位置。这是由于序列被填充到统一长度时填充了许多占位符，其并不包含有效信息。由于应用了分组策略处理，填充位与分类向量的注意力分数也呈现了块状分布的特征。

（4）综合结果验证了序列长度缩减策略的有效性。

5.6 本章小结

本章为实验设计与分析的独立章节，内容主要围绕实验展开。首先介绍了本文的相关实验设置，包括数据集、基准模型和参数设置等。其次，针对自然语言处理领域的两个不同任务在多个数据集上进行了实验，以此来对模型的效果和效率进行比较，实验结果证实了本文模型方案的有效性和高效性，同时也体现了在收敛速度上的优势。此外，本文通过消融实验验证了不同模块对模型的提升作用，结果表明相关策略均对模型效果或效率的提升有一定促进作用。本文还进行了参数实验，探究了若干关键参数设置对模型效果和效率的影响，以此得出较为推荐的参数设置范围。最后，通过可视化实验验证了本文先前的假设，同时探究了分类向量的作用原理。综合各个实验模块的结果，本文提出的相关方法理论对模型效果和模型效率均有一定的提升作用。通过设置部分参数，还可以根据实际应用场景的需求灵活地调整对模型效果和效率的侧重。

第 6 章 总结与展望

6.1 本文总结

随着深度学习技术的快速发展和互联网数据量的飞速膨胀，深度神经网络的规模越来越大、参数量也越来越多。深层的网络结构和大量的参数使得模型能从数据中学到更丰富、更高层的信息，同时也要求更强的计算能力和更高的存储能力。在硬件发展接近突破摩尔定律的趋势下，如何从模型结构层面入手进行改造优化、提高模型效率是一个关键的问题。本文聚焦于 Transformer 架构为代表的深度学习模型，从横向层内和纵向层间两个宏观优化视角开展研究，提出了以层内分组和层间选择为核心的若干优化策略，在保证模型效果的同时，减少了模型的内存占用、提高了模型训练和推理过程的速度。本文的主要工作总结如下：

（1）在横向层内，使用了基于内容的稀疏注意力机制。为了提高分组的准确性，使用了距离信息、预训练词向量作为分组阶段的计算依据。同时提出了基于 K-Means++的采样生成算法和基于距离的平均生成算法以优化聚类中心的初始化过程，提高了分组前期工作的准确性。在分组后期，为了弥补基于注意力进行分组的误差，提出了额外的分组损失训练任务和聚类中心信息增强策略，提高了分组策略的稳定性。针对分组注意力模型在分类任务中的应用问题，设计了全局分类信息层次融合的方法，更有效、更合理地获取输入序列整体的信息，优化模型在分类场景下的效果。

（2）在纵向层间，设计了基于注意力分数矩阵的贡献度计算方法。基于贡献度量化指标提出了两种应用策略：通过选择各组代表提出了全局信息引入策略，融合局部和全局视野信息，缓解了分组注意力带来的信息交互损失问题。通过选择序列中重要的 Token，提出了序列长度缩减策略应用于训练和推理阶段，在尽可能保证模型效果的前提下，进一步加快模型运行速度、提高模型效率。

（3）实验结果表明，本文的模型在多个数据集上的效果和效率普遍优于当前的基线模型。消融实验的结果证实了本文相关策略和方法的有效性。参数实验和可视化实验帮助更好地理解模型的作用原理。

6.2 未来展望

随着 ChatGPT 引领的新一波大模型发展趋势，对于深度学习模型高性能计算的需求会更加强烈，对该问题的研究在可预见的未来也将会持续下去。基于本文的相关工作和对当前研究现状的认知，后续还可从如下几个方向进行进一步深入研究：

（1）模型效率优化与目标硬件协同发展、深度融合。目前的大多数硬件加速器面向通用场景进行设计，提供的是更细粒度的底层计算单元。现如今深度学习领域的许多模型都参考了 Transformer 的结构，使用注意力计算模块。因此，面向软件层面的特定计算结构，在硬件上设计定制的高效计算单元，软件和硬件进行协同设计，通过更加紧密的软硬件继承可以实现更高效的计算。

（2）稀疏并且高效的注意力计算研究。针对 Transformer 模型效率优化问题，稀疏注意力是目前常见的解决方案，基于内容的稀疏注意力又是相对更合理的策略。然而，信息交互的损失是该策略不可避免的一个问题。通过对最大内积搜索、信息交互弥补等问题的进一步研究，或许可以得到更加稀疏且高效的注意力计算模式。

（3）应用场景拓展。目前基于内容的稀疏注意力已经应用于语言模型和图像分类等领域，未来可以探索它在其他场景下的应用，例如机器翻译、智能问答和语音识别等领域。序列长度缩减策略由于对输入序列的改变，暂时也只能在分类场景中直接应用，如何将其思想迁移到非分类任务场景（如语言模型、机器翻译等）也是后续值得研究的一个问题。

参考文献

[1] Vaswani A, Shazeer N, Parmar N, et al. Attention is All you Need[C]//Guyon I, Luxburg U von, Bengio S, et al. Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA. 2017: 5998-6008.

[2] Devlin J, Chang M W, Lee K, et al. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding[C]//Burstein J, Doran C, Solorio T. Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers). Association for Computational Linguistics, 2019: 4171-4186.

[3] Brown T B, Mann B, Ryder N, et al. Language Models are Few-Shot Learners[C]//Larochelle H, Ranzato M, Hadsell R, et al. Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual. 2020.

[4] Raffel C, Shazeer N, Roberts A, et al. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer[J]. J. Mach. Learn. Res., 2020, 21: 140:1-140:67.

[5] Dosovitskiy A, Beyer L, Kolesnikov A, et al. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale[C]//9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021. OpenReview.net, 2021.

[6] Bochkovskiy A, Wang C Y, Liao H Y M. YOLOv4: Optimal Speed and Accuracy of Object Detection[OL]. CoRR, 2020, abs/2004.10934. https://arxiv.org/abs/2004.10934.

[7] Liu Z, Lin Y, Cao Y, et al. Swin Transformer: Hierarchical Vision Transformer using Shifted Windows[C]//2021 IEEE/CVF International Conference on Computer Vision, ICCV 2021, Montreal, QC, Canada, October 10-17, 2021. IEEE, 2021: 9992-10002.

[8] Baevski A, Zhou Y, Mohamed A, et al. wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations[C]//Larochelle H, Ranzato M, Hadsell R, et al. Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual. 2020.

[9] Gulati A, Qin J, Chiu C C, et al. Conformer: Convolution-augmented Transformer for Speech Recognition[C]//Meng H, Xu B, Zheng T F. Interspeech 2020, 21st Annual Conference of the International Speech Communication Association, Virtual Event, Shanghai, China, 25-29 October 2020. ISCA, 2020: 5036-5040.

[10] Fedus W, Zoph B, Shazeer N. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity[J]. CoRR, 2021, abs/2101.03961. https://arxiv.org/abs/2101.03961.

[11] Zhang Q, Zuo S, Liang C, et al. PLATON: Pruning Large Transformer Models with Upper Confidence Bound of Weight Importance[C]//Chaudhuri K, Jegelka S, Song L, et al. International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA: 卷 162. PMLR, 2022: 26809-26823.

[12] Wang Z J, Turko R, Chau D H. Dodrio: Exploring Transformer Models with Interactive Visualization[C]//Ji H, Park J C, Xia R. Proceedings of the Joint Conference of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing, ACL 2021 - System Demonstrations, Online, August 1-6, 2021. Association for Computational Linguistics, 2021: 132-141.

[13] Hao Y, Dong L, Wei F, et al. Self-Attention Attribution: Interpreting Information Interactions Inside Transformer[C]//Thirty-Fifth AAAI Conference on Artificial Intelligence, AAAI 2021, Thirty-Third Conference on Innovative Applications of Artificial Intelligence, IAAI 2021, The Eleventh Symposium on Educational Advances in Artificial Intelligence, EAAI 2021, Virtual Event, February 2-9, 2021. AAAI Press, 2021: 12963-12971.

[14] Manakul P, Gales M J F. Long-Span Summarization via Local Attention and Content Selection[C]//Zong C, Xia F, Li W, et al. Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing, ACL/IJCNLP 2021, (Volume 1: Long Papers), Virtual Event, August 1-6, 2021. Association for Computational Linguistics, 2021: 6026-6041.

[15] Beltagy I, Peters M E, Cohan A. Longformer: The long-document transformer[J]. arXiv preprint arXiv:2004.05150, 2020.

[16] Zaheer M, Guruganesh G, Dubey K A, et al. Big Bird: Transformers for Longer Sequences[C]//Larochelle H, Ranzato M, Hadsell R, et al. Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual. 2020.

[17] Kitaev N, Kaiser L, Levskaya A. Reformer: The Efficient Transformer[C]//8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020.

[18] Roy A, Saffar M, Vaswani A, et al. Efficient Content-Based Sparse Attention with Routing Transformers[J]. Trans. Assoc. Comput. Linguistics, 2021, 9: 53-68.

[19] Wang N, Gan G, Zhang P, et al. ClusterFormer: Neural Clustering Attention for Efficient and Effective Transformer[C]//Muresan S, Nakov P, Villavicencio A. Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2022, Dublin, Ireland, May 22-27, 2022. Association for Computational Linguistics, 2022: 2390-2402.

[20] Wang S, Li B Z, Khabsa M, et al. Linformer: Self-attention with linear complexity[J]. arXiv preprint arXiv:2006.04768, 2020.

[21] Lee-Thorp J, Ainslie J, Eckstein I, et al. FNet: Mixing Tokens with Fourier Transforms[C]//Carpuat M, Marneffe M C de, Ruíz I V M. Proceedings of the 2022

Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL 2022, Seattle, WA, United States, July 10-15, 2022. Association for Computational Linguistics, 2022: 4296-4313.

[22] Liu Z, Sun M, Zhou T, et al. Rethinking the Value of Network Pruning[C]//7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019.

[23] Frankle J, Carbin M. The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks[C]//7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019.

[24] 吴蔚. 面向神经网络加速的硬件友好型模型压缩方法研究[D].南京大学,2021.DOI:10.27235/d.cnki.gnjiu.2021.001917.

[25] Micikevicius P, Narang S, Alben J, et al. Mixed Precision Training[C]//6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings.

[26] Hoffer E, Banner R, Golan I, et al. Norm matters: efficient and accurate normalization schemes in deep networks[C]//Bengio S, Wallach H M, Larochelle H, et al. Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3-8, 2018, Montréal, Canada. 2018: 2164-2174.

[27] 黄一凡. 深度神经网络的压缩和性能提升研究[D].中国科学技术大学,2022.DOI:10.27517/d.cnki.gzkju.2022.001211.

[28] Zhang D, Yang J, Ye D, et al. LQ-Nets: Learned Quantization for Highly Accurate and Compact Deep Neural Networks[C]//Ferrari V, Hebert M, Sminchisescu C, et al. Computer Vision - ECCV 2018 - 15th European Conference, Munich, Germany, September 8-14, 2018, Proceedings, Part VIII: 卷 11212. Springer, 2018: 373-390.

[29] Sanh V, Debut L, Chaumond J, et al. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter[J]. CoRR, 2019, abs/1910.01108. http://arxiv.org/abs/1910.01108.

[30] Arthur D, Vassilvitskii S. k-means++: the advantages of careful seeding[C]//Bansal N, Pruhs K, Stein C. Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2007, New Orleans, Louisiana, USA, January 7-9, 2007. SIAM, 2007: 1027-1035.

[31] 姜云桥. 基于新一代申威众核处理器的 Transformer 模型并行优化的研究[D]. 华东师范大学, 2022[2023-03-28]. DOI:10.27149/d.cnki.ghdsu.2022.002946.

[32] Chen T, Moreau T, Jiang Z, et al. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning[C]//Arpaci-Dusseau A C, Voelker G. 13th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2018, Carlsbad, CA, USA, October 8-10, 2018. USENIX Association, 2018: 578-594.

[33] Mikolov T, Sutskever I, Chen K, et al. Distributed Representations of Words and Phrases and their Compositionality[C]//Burges C J C, Bottou L, Ghahramani Z, et al. Advances in Neural Information Processing Systems 26: 27th Annual Conference on Neural Information Processing Systems 2013. Proceedings of a meeting held December 5-8, 2013, Lake Tahoe, Nevada, United States. 2013: 3111-3119.

[34] Pennington J, Socher R, Manning C D. Glove: Global Vectors for Word Representation[C]//Moschitti A, Pang B, Daelemans W. Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, EMNLP 2014, October 25-29, 2014, Doha, Qatar, A meeting of SIGDAT, a Special Interest Group of the ACL. ACL, 2014: 1532-1543.

[35] Modarressi A, Mohebbi H, Pilehvar M T. AdapLeR: Speeding up Inference by Adaptive Length Reduction[C]//Muresan S, Nakov P, Villavicencio A. Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2022, Dublin, Ireland, May 22-27, 2022. Association for Computational Linguistics, 2022: 1-15.

[36] Zhao J, Wang Y, Bao J, et al. Fine- and Coarse-Granularity Hybrid Self-Attention for Efficient BERT[C]//Muresan S, Nakov P, Villavicencio A. Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2022, Dublin, Ireland, May 22-27, 2022. Association for Computational Linguistics, 2022: 4811-4820.

[37] Goyal S, Choudhury A R, Raje S, et al. PoWER-BERT: Accelerating BERT Inference via Progressive Word-vector Elimination[C]//Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event: 卷 119. PMLR, 2020: 3690-3699.

[38] Ye D, Lin Y, Huang Y, et al. TR-BERT: Dynamic Token Reduction for Accelerating BERT Inference[C]//Toutanova K, Rumshisky A, Zettlemoyer L, et al. Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2021, Online, June 6-11, 2021. Association for Computational Linguistics, 2021: 5798-5809.

[39] Pietruszka M, Borchmann L, Garncarek L. Sparsifying Transformer Models with Trainable Representation Pooling[C]//Muresan S, Nakov P, Villavicencio A. Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2022, Dublin, Ireland, May 22-27, 2022. Association for Computational Linguistics, 2022: 8616-8633.

[40] Lang K, 1995. Newsweeder: Learning to filter netnews[C]//Proceedings of the Twelfth International Conference on Machine Learning. 331-339.

[41] Pang B, Lee L. A Sentimental Education: Sentiment Analysis Using Subjectivity Summarization Based on Minimum Cuts[C]//Proceedings of the 42nd Annual Meeting on Association for Computational Linguistics. USA: Association for Computational Linguistics, 2004: 271-es.

[42] Maas A L, Daly R E, Pham P T, et al. Learning Word Vectors for Sentiment Analysis[C]//Lin D, Matsumoto Y, Mihalcea R. The 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies, Proceedings of the Conference, 19-24 June, 2011, Portland, Oregon, USA. The Association for Computer Linguistics, 2011: 142-150.

[43] Mohammad S, Bravo-Marquez F, Salameh M, et al. Semeval-2018 task 1: Affect in tweets[C]//Proceedings of the 12th international workshop on semantic evaluation. 2018: 1-17.

[44] Wu F, Qiao Y, Chen J H, et al. MIND: A Large-scale Dataset for News Recommendation[C]//Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics. Online: Association for Computational Linguistics, 2020: 3597-3606.

[45] Tjong Kim Sang E F, De Meulder F. Introduction to the CoNLL-2003 Shared Task: Language-Independent Named Entity Recognition[C]//Proceedings of the Seventh Conference on Natural Language Learning at HLT-NAACL 2003. 2003: 142-147.

[46] Paszke A, Gross S, Massa F, et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library[C]//Wallach H M, Larochelle H, Beygelzimer A, et al. Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada. 2019: 8024-8035.

致谢

随着毕业论文工作接近尾声，也马上要对在武大的六年学习生涯说再见了。在这几年的时光里，有上课时的求知若渴和昏昏欲睡；有宿舍游戏的笑声和期末复习时的翻书声；有看论文时的苦闷，也有实现想法后的欣喜……这些点点滴滴构成了最珍贵的回忆。在即将步入社会进行角色转变的时机，回望这段日子，有努力过、有收获过、有成长过。

在学业上，首先要感谢我的导师彭敏教授，一直以来都很尊重我的研究想法，在我遇到问题时总能提供很及时的帮助。感谢老师提供的自由、舒适的学习氛围和环境，让我在实验室学习都有满满的幸福感。感谢辅导员张老师、于老师，教学秘书肖老师，感谢他们在课程和学习生活上提供的帮助。感谢贾旭博士，他就像实验室大家长，在每天到实验室时总会在那里用手势比出一个大大的“赞”，同时也感谢他对毕业论文提出的宝贵的修改意见。感谢刘奔博士，不仅对实验室日常事务提供了很多支持，同时也是实验室为数不多的球友。感谢徐文杰博士、彭美政、彭淼、杨世杰和陈嘉辉，一起度过了健身、烧烤、约饭等快乐时光。还要感谢已经毕业的张鼎、谢铄圻、袁尤良、胡宇翔等师兄，在研究生入学初期提供了很多宝贵经验，现在也还在提供工作经验。要感谢的人太多，总之很幸运遇到实验室的小伙伴。

在生活上，感谢舍友之间的互相扶持和帮助。感谢杨世杰、封奇志，作为本科就熟识的老朋友一直都互相陪伴。感谢姬一峰，带我培养摄影的爱好，一起吃饭、聊电子产品、聊人生话题。最重要的，要感谢我的家人，是他们一直默默地支持我，每次视频通话都会提醒我多吃点好的、早点睡觉，他们是最牵挂我的人。

最后，还要感谢各位专家老师抽出宝贵的时间来评审我的论文。

希望以后的自己能够永葆一颗少年的心，勇敢大胆地向前冲吧！

附录 A：在学期间参与的科研项目

[1] 基于多范式编程的异构框架智能计算平台，湖北省重点研发计划，核心成员.

[2] 智慧城管-城管应急管理平台，武汉市城管委合作项目，核心成员.

[3] CAAI 华为-人工智能学会合作基金，核心成员.

[4] 钉钉高效会议工具研发，阿里云计算有限公司合作项目，参与人.

武汉大学学位论文使用授权协议书

本学位论文作者愿意遵守武汉大学关于保存、使用学位论文的管理办法及规定，即：学校有权保存学位论文的印刷本和电子版，并提供文献检索与阅览服务；学校可以采用影印、缩印、数字化或其它复制手段保存论文；在以教学与科研服务为目的前提下，学校可以在校园网内公布部分或全部内容。

一、在本论文提交当年，同意在校园网内以及中国高等教育文献保障系统（CALIS）、高校学位论文系统提供查询及前十六页浏览服务。

二、在本论文提交□当年/□一年/□两年/□三年以后，同意在校园网内允许读者在线浏览并下载全文，学校可以为存在馆际合作关系的兄弟高校用户提供文献传递服务和交换服务。（保密论文解密后遵守此规定）

论文作者（签名）：_____

学　　号：_____

学　　院：_____

日期：　　年　　月　　日